

Metal Shading Language Specification

Version 1.2

1	Introduction.....	6
1.1	Audience.....	6
1.2	Organization of this Specification.....	6
1.3	References.....	6
1.4	Metal and C++14.....	7
1.4.1	Overloading.....	7
1.4.2	Templates.....	7
1.4.3	Preprocessing Directives.....	7
1.4.4	Restrictions.....	7
1.5	Metal Pixel Coordinate System.....	8
2	Data Types.....	9
2.1	Scalar Data Types.....	9
2.2	Vector and Matrix Data Types.....	10
2.2.1	Accessing Vector Components.....	10
2.2.2	Accessing Matrix Components.....	13
2.2.3	Vector Constructors.....	13
2.2.4	Matrix Constructors.....	14
2.3	Atomic Data Types.....	15
2.4	Buffers.....	15
2.5	Textures.....	16
2.6	Samplers.....	17
2.7	Arrays and Structs.....	20
2.7.1	Arrays of Textures.....	21
2.7.1.1	Element Access.....	21
2.7.1.2	Capacity.....	21
2.7.1.3	Element Access.....	22
2.7.1.4	Capacity.....	22
2.7.1.5	Constructors.....	22
2.8	Alignment of Types.....	23
2.9	Packed Vector Data Types.....	26
2.10	Implicit Type Conversions.....	27
2.11	Type Conversions and Re-interpreting Data.....	28
3	Operators.....	30
3.1	Scalar and Vector Operators.....	30
3.2	Matrix Operators.....	32

4	Functions, Variables, and Qualifiers	35
4.1	Function Qualifiers	35
4.1.1	Post-Tessellation Vertex Function	35
4.1.1.1	Patch Type and Number of Control Points Per-Patch.....	36
4.1.2	Attribute Qualifiers for Fragment Functions	36
4.2	Address Space Qualifiers for Variables and Arguments.....	37
4.2.1	device Address Space	37
4.2.2	threadgroup Address Space	38
4.2.3	constant Address Space.....	39
4.2.4	thread Address Space	39
4.3	Function Arguments and Variables	40
4.3.1	Attribute Qualifiers to Locate Buffers, Textures and Samplers	40
4.3.1.1	Vertex function example that specifies resources and outputs to device memory	42
4.3.2	Struct of buffers and textures	43
4.3.3	Attribute Qualifiers to Locate Per-Vertex Inputs	45
4.3.4	Attribute Qualifiers for Built-in Variables	48
4.3.4.1	Attribute Qualifiers for Vertex Function Input	48
4.3.4.2	Attribute Qualifiers for Post-Tessellation Vertex Function Input	49
4.3.4.3	Attribute Qualifiers for Vertex Function Output	49
4.3.4.4	Attribute Qualifiers for Fragment Function Input	50
4.3.4.5	Attribute Qualifiers for Fragment Function Output.....	51
4.3.4.6	Attribute Qualifiers for Kernel Function Input	53
4.3.5	stage_in Qualifier	55
4.3.5.1	Vertex function example that uses the stage_in qualifier	55
4.3.5.2	Fragment function example that uses the stage_in qualifier	57
4.3.5.3	Kernel function example that uses the stage_in qualifier	58
4.4	Storage Class Specifiers.....	59
4.5	Sampling and Interpolation Qualifiers.....	60
4.6	Per-Fragment Function vs. Per-Sample Function.....	61
4.7	Programmable Blending	61
4.8	Graphics Function – Signature Matching	62
4.8.1	Vertex – Fragment Signature Matching	62
4.9	Program Scope Function Constants	68
4.9.1	Specifying program scope function constants	69
4.9.1.1	Using function constants to control code paths to compile	70

4.9.1.2	Using function constants when declaring the arguments of functions	71
4.9.1.3	Using function constants for elements of a <code>[[stage_in]]</code> struct	74
4.10	Additional Restrictions	76
5	Metal Standard Library	77
5.1	Namespace and Header Files	77
5.2	Common Functions	77
5.3	Integer Functions	78
5.4	Relational Functions	80
5.5	Math Functions	81
5.6	Matrix Functions	85
5.7	Geometric Functions	85
5.8	Compute Functions	86
5.8.1	Threadgroup Synchronization Functions	86
5.9	Graphics Functions	88
5.9.1	Fragment Functions	88
5.9.1.1	Fragment Functions – Derivatives	88
5.9.1.2	Fragment Functions – Samples	89
5.9.1.3	Fragment Functions – Flow Control	89
5.10	Texture Functions	90
5.10.1	1D Texture	90
5.10.2	1D Texture Array	91
5.10.3	2D Texture	91
5.10.3.1	2D Texture Sampling Example	92
5.10.4	2D Texture Array	93
5.10.5	3D Texture	94
5.10.6	Cube Texture	94
5.10.7	Cube Array Texture	96
5.10.8	2D Multisampled Texture	97
5.10.9	2D Depth Texture	97
5.10.10	2D Depth Texture Array	98
5.10.11	Cube Depth Texture	99
5.10.12	Cube Array Depth Texture	100
5.10.13	2D Multisampled Depth Texture	102
5.10.14	Texture Fence Functions	102
5.10.15	Null Texture Functions	102

5.11	Pack and Unpack Functions	103
5.11.1	Unpack Integer(s); Convert to a Floating-Point Vector	103
5.11.2	Convert Floating-Point Vector to Integers, then Pack the Integers	104
5.12	Atomic Functions.....	105
5.12.1	Atomic Store Functions	105
5.12.2	Atomic Load Functions	105
5.12.3	Atomic Exchange Functions	106
5.12.4	Atomic Compare and Exchange functions.....	106
5.12.5	Atomic Fetch and Modify functions.....	107
6	Compiler Options	108
6.1	Pre-Processor Options	108
6.2	Math Intrinsics Options.....	108
6.3	Options Controlling the Language Version	109
6.4	Options to Request or Suppress Warnings	109
7	Numerical Compliance	110
7.1	INF, NaN and Denormalized Numbers	110
7.2	Rounding Mode	110
7.3	Floating-point Exceptions.....	110
7.4	Relative Error as ULPs.....	110
7.5	Edge Case Behavior in Flush To Zero Mode	114
7.6	Conversion Rules for Floating-point and Integer Types.....	114
7.7	Texture Addressing and Conversion Rules.....	115
7.7.1	Conversion rules for normalized integer pixel data types	115
7.7.1.1	Converting normalized integer pixel data types to floating-point values	115
7.7.1.2	Converting floating-point values to normalized integer pixel data types.....	116
7.7.2	Conversion rules for half precision floating-point pixel data type	117
7.7.3	Conversion rules for single precision floating-point pixel data type	117
7.7.4	Conversion rules for 11-bit and 10-bit floating-point pixel data type	117
7.7.5	Conversion rules for 9-bit floating-point pixel data type with a 5-bit shared exponent	118
7.7.6	Conversion rules for signed and unsigned integer pixel data types.....	119
7.7.7	Conversion rules for sRGBA and sBGRA Textures	119
8	Revision History	121

1 Introduction

This document describes the Metal Unified Graphics and Compute Language. Metal is a C++ based programming language that developers can use to write code that is executed on the GPU for graphics and general-purpose data-parallel computations. Since Metal is based on C++, developers will find it familiar and easy to use. With Metal, both graphics and compute programs can be written with a single, unified language, which allows tighter integration between the two.

Metal is designed to work together with the Metal framework, which manages the execution, and optionally the compilation, of Metal code. Metal uses clang and LLVM so developers get a compiler that delivers close to the metal performance for code executing on the GPU.

1.1 Audience

Developers who are writing code with the Metal framework will want to read this document, because they will need to use the Metal language to write graphics and compute programs to be executed on the GPU.

1.2 Organization of this Specification

This document is organized into the following chapters:

- This chapter, “Introduction,” is an introduction to this document and covers the similarities and differences between Metal and C++14.
- “Data Types” lists the Metal data types, including types that represent vectors, matrices, buffers, textures, and samplers. It also discusses type alignment and type conversion.
- “Operators” lists the Metal operators.
- “Functions, Variables, and Qualifiers” details how functions and variables are declared, sometimes with qualifiers that restrict how they are used.
- “Metal Standard Library” defines a collection of built-in Metal functions.
- “Compiler Options” details the options for the Metal compiler, including pre-processor directives, options for math intrinsics, and options that control optimization.
- “Numerical Compliance” describes requirements for representing floating-point numbers, including accuracy in mathematical operations.

1.3 References

C++14

Stroustrup, Bjarne. *The C++ Programming Language*. Harlow: Addison-Wesley, 2013.

Metal

The *Metal Programming Guide* provides a detailed introduction to writing apps with the Metal framework.

The *Metal Framework Reference* details individual classes in the Metal framework.

1.4 Metal and C++14

The Metal programming language is based on the C++14 Specification (a.k.a., the ISO/IEC JTC1/SC22/WG21 N4431 Language Specification) with specific extensions and restrictions. Please refer to the C++14 Specification for a detailed description of the language grammar.

This section and its subsections describe modifications and restrictions to the C++14 language supported in Metal.

For more information about Metal pre-processing directives and compiler options, see Chapter 6 of this document.

1.4.1 Overloading

Metal supports overloading as defined by section 13 of the C++14 Specification. The function overloading rules are extended to include the address space qualifier of an argument. Metal graphics and kernel functions cannot be overloaded. (For definition of graphics and kernel functions, see section 4.1 of this document.)

1.4.2 Templates

Metal supports templates as defined by section 14 of the C++14 Specification.

1.4.3 Preprocessing Directives

Metal supports the pre-processing directives defined by section 16 of the C++14 Specification.

1.4.4 Restrictions

The following C++14 features are not available in Metal (section numbers in this list refer to the C++14 Specification):

- lambda expressions (section 5.1.2)
- `dynamic_cast` operator (section 5.2.7)
- type identification (section 5.2.8)
- recursive function calls (section 5.2.2, item 9)
- `new` and `delete` operators (sections 5.3.4 and 5.3.5)
- `noexcept` operator (section 5.3.7)
- `goto` statement (section 6.6)
- `register`, `thread_local` storage qualifiers (section 7.1.1)
- virtual function qualifier (section 7.1.2)
- derived classes (section 10, section 11)
- exception handling (section 15)

The C++ standard library must **not** be used in Metal code. Instead of the C++ standard library, Metal has its own standard library that is discussed in Chapter 5 of this document.

Metal restricts the use of pointers:

- Arguments to Metal graphics and kernel functions declared in a program that are pointers must be declared with the Metal `device`, `threadgroup` or `constant` address space qualifier. (See section 4.2 of this document for more about Metal address space qualifiers.)
- Function pointers are not supported.

A Metal function cannot be called `main`.

1.5 Metal Pixel Coordinate System

In Metal, the origin of the pixel coordinate system of a framebuffer attachment is defined at the top left corner. Similarly, the origin of the pixel coordinate system of a framebuffer attachment is the top left corner.

2 Data Types

This chapter details the Metal data types, including types that represent vectors and matrices. Atomic data types, buffers, textures, samplers, arrays, and user-defined structs are also discussed. Type alignment and type conversion are also described.

2.1 Scalar Data Types

Metal supports the scalar types listed in Table 1. Metal does **not** support the `double`, `long`, `unsigned long`, `long long`, `unsigned long long`, and `long double` data types.

Table 1 Metal Scalar Data Types

Type	Description
<code>bool</code>	A conditional data type that has the value of either <code>true</code> or <code>false</code> . The value <code>true</code> expands to the integer constant 1, and the value <code>false</code> expands to the integer constant 0.
<code>char</code> <code>int8_t</code>	A signed two's complement 8-bit integer.
<code>unsigned char</code> <code>uchar</code>	An unsigned 8-bit integer.
<code>short</code>	A signed two's complement 16-bit integer.
<code>unsigned short</code> <code>ushort</code>	An unsigned 16-bit integer.
<code>int</code>	A signed two's complement 32-bit integer.
<code>unsigned int</code> <code>uint</code>	An unsigned 32-bit integer.
<code>half</code>	A 16-bit floating-point. The half data type must conform to the IEEE 754 binary16 storage format.
<code>float</code>	A 32-bit floating-point. The float data type must conform to the IEEE 754 single precision storage format.
<code>size_t</code>	An unsigned integer type of the result of the <code>sizeof</code> operator. This is a 64-bit unsigned integer.
<code>ptrdiff_t</code>	A signed integer type that is the result of subtracting two pointers. This is a 64-bit signed integer.
<code>void</code>	The <code>void</code> type comprises an empty set of values; it is an incomplete type that cannot be completed.

NOTE: Metal supports the standard **f** or **F** suffix to specify a single precision floating-point literal value (e.g., **0.5f** or **0.5F**). In addition, Metal supports the **h** or **H** suffix to specify a half precision floating-point literal value (e.g., **0.5h** or **0.5H**). Metal also supports the **u** or **U** suffix for unsigned integer literals.

2.2 Vector and Matrix Data Types

Metal supports a subset of the vector and matrix data types implemented by the system vector math library.

The vector type names supported are:

```
booln,  
charn, shortn, intn, uchar, ushortn, uintn,  
halfn and floatn
```

n is 2, 3, or 4 representing a 2-, 3- or 4- component vector type.

The matrix type names supported are:

```
halfnxm and floatnxm
```

where *n* and *m* are number of columns and rows. *n* and *m* can be 2, 3 or 4. A matrix of type `floatnxm` is composed of *n* `floatm` vectors. Similarly, a matrix of type `halfnxm` is composed of *n* `halfm` vectors.

2.2.1 Accessing Vector Components

Vector components can be accessed using an array index. Array index 0 refers to the first component of the vector, index 1 to the second component, and so on. The following examples show various ways to access array components:

```
pos = float4(1.0f, 2.0f, 3.0f, 4.0f);  
  
float x = pos[0]; // x = 1.0  
float z = pos[2]; // z = 3.0  
  
float4 vA = float4(1.0f, 2.0f, 3.0f, 4.0f);  
float4 vB;  
  
for (int i=0; i<4; i++)  
    vB[i] = vA[i] * 2.0f // vB = (2.0, 4.0, 6.0, 8.0);
```

Metal supports using the period (`.`) as a selection operator to access vector components, using letters that may indicate coordinate or color data:

```
<vector_data_type>.xyzw or  
<vector_data_type>.rgba
```

In the following code, the vector `test` is initialized, and then components are accessed using the `.xyzw` or `.rgba` selection syntax:

```
int4 test = int4(0, 1, 2, 3);
int a = test.x; // a = 0
int b = test.y; // b = 1
int c = test.z; // c = 2
int d = test.w; // d = 3
int e = test.r; // e = 0
int f = test.g; // f = 1
int g = test.b; // g = 2
int h = test.a; // h = 3
```

The component selection syntax allows multiple components to be selected.

```
float4 c;
c.xyzw = float4(1.0f, 2.0f, 3.0f, 4.0f);
c.z = 1.0f;
c.xy = float2(3.0f, 4.0f);
c.xyz = float3(3.0f, 4.0f, 5.0f);
```

The component selection syntax also allows components to be permuted or replicated.

```
float4 pos = float4(1.0f, 2.0f, 3.0f, 4.0f);
float4 swiz = pos.wzyx; // swiz = (4.0f, 3.0f, 2.0f, 1.0f)
float4 dup = pos.xxyy; // dup = (1.0f, 1.0f, 2.0f, 2.0f)
```

The component group notation can occur on the left hand side of an expression. To form the lvalue, swizzling may be applied. The resulting lvalue may be either the scalar or vector type, depending on number of components specified. Each component must be a supported scalar or vector type. The resulting lvalue of vector type must not contain duplicate components.

```
float4 pos = float4(1.0f, 2.0f, 3.0f, 4.0f);
// pos = (5.0, 2.0, 3.0, 6.0)
pos.xw = float2(5.0f, 6.0f);

// pos = (8.0, 2.0, 3.0, 7.0)
pos.wx = float2(7.0f, 8.0f);

// pos = (3.0, 5.0, 9.0, 7.0)
```

```
pos.xyz = float3(3.0f, 5.0f, 9.0f);
```

The following methods of vector component access are not permitted and result in a compile-time error:

- Accessing components beyond those declared for the vector type is an error. 2-component vector data types can only access `.xy` or `.rg` elements. 3-component vector data types can only access `.xyz` or `.rgb` elements. For instance:

```
float2 pos;  
pos.x = 1.0f; // is legal; so is y  
pos.z = 1.0f; // is illegal; so is w
```

```
float3 pos;  
pos.z = 1.0f; // is legal  
pos.w = 1.0f; // is illegal
```

- Accessing the same component twice on the left-hand side is ambiguous; for instance,
// illegal – 'x' used twice
`pos.xx = float2(3.0f, 4.0f);`

```
// illegal – mismatch between float2 and float4  
pos.xy = float4(1.0f, 2.0f, 3.0f, 4.0f);
```

- The `.rgba` and `.xyzw` qualifiers cannot be intermixed in a single access; for instance,
`float4 pos = float4(1.0f, 2.0f, 3.0f, 4.0f);`

```
pos.x = 1.0f;      // OK  
pos.g = 2.0f;      // OK  
pos.xg = float2(3.0f, 4.0f); // illegal – mixed qualifiers used  
float3 coord = pos.ryz; // illegal – mixed qualifiers used
```

- A pointer or reference to a vector with swizzles; for instance

```
float4 pos = float4(1.0f, 2.0f, 3.0f, 4.0f);  
my_func(&pos.xy); // illegal
```

The `sizeof` operator on a vector type returns the size of the vector, which is given as the number of components * size of each component. For example, `sizeof(float4)` returns 16 and `sizeof(half4)` returns 8.

2.2.2 Accessing Matrix Components

The `floatn $\times$ m` and `halfn $\times$ m` matrices can be accessed as an array of `n float $\times$ m` or `n half $\times$ m` entries.

The components of a matrix can be accessed using the array subscripting syntax. Applying a single subscript to a matrix treats the matrix as an array of column vectors. The top column is column 0. A second subscript would then operate on the resulting vector, as defined earlier for vectors. Hence, two subscripts select a column and then a row.

```
float4x4 m;

// sets the 2nd column to all 2.0
m[1] = float4(2.0f);

// sets the 1st element of the 1st column to 1.0
m[0][0] = 1.0f;

// sets the 4th element of the 3rd column to 3.0
m[2][3] = 3.0f;
```

Accessing a component outside the bounds of a matrix with a non-constant expression results in undefined behavior. Accessing a matrix component that is outside the bounds of the matrix with a constant expression generates a compile-time error.

2.2.3 Vector Constructors

Constructors can be used to create vectors from a set of scalars or vectors. When a vector is initialized, its parameter signature determines how it is constructed. For instance, if the vector is initialized with only a single scalar parameter, all components of the constructed vector are set to that scalar value.

If a vector is constructed from multiple scalars, one or more vectors, or a mixture of these, the vector's components are constructed in order from the components of the arguments. The arguments are consumed from left to right. Each argument has all its components consumed, in order, before any components from the next argument are consumed.

This is a complete list of constructors that are available for `float4`:

```
float4(float x);
float4(float x, float y, float z, float w);
float4(float2 a, float2 b);
float4(float2 a, float b, float c);
float4(float a, float b, float2 c);

float4(float a, float2 b, float c);
float4(float3 a, float b);
float4(float a, float3 b);
float4(float4 x);
```

This is a complete list of constructors that are available for `float3`:

```
float3(float x);
float3(float x, float y, float z);
float3(float a, float2 b);
float3(float2 a, float b);
float3(float3 x);
```

This is a complete list of constructors that are available for `float2`:

```
float2(float x);
float2(float x, float y);
float2(float2 x);
```

The following examples illustrate uses of the constructors:

```
float x = 1.0f, y = 2.0f, z = 3.0f, w = 4.0f;
float4 a = float4(0.0f);
float4 b = float4(x, y, z, w);
float2 c = float2(5.0f, 6.0f);

float2 a = float2(x, y);
float2 b = float2(z, w);
float4 x = float4(a.xy, b.xy);
```

Under-initializing a vector constructor is a compile-time error.

2.2.4 Matrix Constructors

Constructors can be used to create matrices from a set of scalars, vectors or matrices. When a matrix is initialized, its parameter signature determines how it is constructed. For example, if a matrix is initialized with only a single scalar parameter, the result is a matrix that contains that scalar for all components of the matrix's diagonal, with the remaining components initialized to 0.0. For example, a call to:

```
float4x4(fval);
```

where `fval` is a scalar floating-point value constructs a matrix with these initial contents:

```
fval  0.0  0.0  0.0
0.0   fval 0.0  0.0
0.0   0.0  fval 0.0
0.0   0.0  0.0  fval
```

A matrix can also be constructed from another matrix that is of the same size, i.e., has the same number of rows and columns. For example,

```
float3x4(float3x4);
float3x4(half3x4);
```

Matrix components are constructed and consumed in column-major order. The matrix constructor must have just enough values specified in its arguments to initialize every component in the constructed matrix object. Providing more arguments than are needed results in an error. Under-initializing a matrix constructor also results in a compile-time error.

A matrix of type `T` with `n` columns and `m` rows can also be constructed from `n` vectors of type `T` with `m` components. The following examples are legal constructors:

```
float2x2(float2, float2);
float3x3(float3, float3, float3);
float3x2(float2, float2, float2);
```

A matrix of type `T` with `n` columns and `m` rows can also be constructed from `n * m` scalars of type `T`. The following examples are legal constructors:

```
float2x2(float, float, float, float);
float3x2(float, float, float, float, float, float);
```

The following are examples of matrix constructors that are **not** supported. A matrix cannot be constructed from combinations of vectors and scalars.

```
// not supported
float2x3(float2 a, float b, float2 c, float d);
```

2.3 Atomic Data Types

The Metal atomic data type is restricted for use by atomic functions implemented by the Metal programming language, as described in section 5.12. These atomic functions are a subset of the C++14 atomic and synchronization functions. Metal atomic functions must operate on Metal atomic data.

The Metal atomic types are defined as:

```
atomic_int and atomic_uint
```

2.4 Buffers

Metal implements buffers as a pointer to a built-in or user defined data type described in the `device` or `constant` address space. (Refer to section 4.2 for a full description of these address qualifiers.). These buffers can be declared in program scope or passed as arguments to a function.

Examples:

```
device float4    *device_buffer;

struct my_user_data {
    float4    a;
    float b;
    int2  c;
};
```

```
constant my_user_data *user_data;
```

2.5 Textures

The texture data type is a handle to one-, two-, or three-dimensional texture data that corresponds to all or a portion of a single mipmap level of a texture. The following templates define specific texture data types:

```
enum class access { sample, read, write, read_write };

texture1d<T, access a = access::sample>
texture1d_array<T, access a = access::sample>
texture2d<T, access a = access::sample>
texture2d_array<T, access a = access::sample>
texture3d<T, access a = access::sample>
texturecube<T, access a = access::sample>
texturecube_array<T, access a = access::sample>
texture2d_ms<T, access a = access::read>
```

Textures with depth formats must be declared as one of the following texture data types:

```
depth2d<T, access a = access::sample>
depth2d_array<T, access a = access::sample>
depthcube<T, access a = access::sample>
depthcube_array<T, access a = access::sample>
depth2d_ms<T, access a = access::read>
```

T specifies the color type returned when reading from a texture or the color type specified when writing to the texture. For texture types (except depth texture types), T can be `half`, `float`, `short`, `ushort`, `int`, or `uint`. For depth texture types, T must be `float`.

NOTE: If T is `int` or `short`, the data associated with the texture must use a signed integer format. If T is `uint` or `ushort`, the data associated with the texture must use an unsigned integer format. If T is `half`, the data associated with the texture must either be a normalized (signed or unsigned integer) or half precision format. If T is `float`, the data associated with the texture must either be a normalized (signed or unsigned integer), half or single precision format.

The access qualifier describes how the texture can be accessed. The supported access qualifiers are:

- `sample` – The texture object can be sampled. `sample` implies the ability to read from a texture with and without a sampler.
- `read` – Without a sampler, a graphics or kernel function can only read the texture object.
- `write` – A graphics or kernel function can write to the texture object.
- `read_write` – A graphics or kernel function can read and write to the texture object.

NOTE:

- For multisampled textures, only the **read** qualifier is supported.
- For depth textures, only the **sample** and **read** qualifiers are supported.

The following example uses these access qualifiers with texture object arguments.

```
void foo (texture2d<float> imgA [[ texture(0) ]],
         texture2d<float, access::read> imgB [[ texture(1) ]],
         texture2d<float, access::write> imgC [[ texture(2) ]])
{
    ...
}
```

(See section 4.3.1 for description of the `texture` attribute qualifier.)

A texture type can also be used as the variable type for any variables declared inside a function. The access qualifier for variables of texture type declared inside a function must be `access::read` or `access::sample`. Declaring variables inside a function to be a texture type but which do not use `access::read` or `access::sample` qualifiers will be a compilation error.

Examples:

```
void foo (texture2d<float> imgA [[ texture(0) ]],
         texture2d<float, access::read> imgB [[ texture(1) ]],
         texture2d<float, access::write> imgC [[ texture(2) ]])
{
    texture2d<float> x = imgA; <– legal
    texture2d<float, access::read> y = imgB; <– legal
    texture2d<float, access::write> z; <– illegal
    ...
}
```

2.6 Samplers

The `sampler` type identifies how to sample a texture. The Metal API allows you to create a sampler object and pass it in an argument to a graphics or kernel function. A sampler object can also be described in the program source instead of in the API. For these cases we only allow a subset of the sampler state to be specified: the addressing mode, filter mode, normalized coordinates and comparison function.

Table 2 describes the list of supported sampler state enums and their associated values (and defaults). These states can be specified when a sampler is initialized in Metal program source.

Table 2 Sampler State Enumeration Values

Enum Name	Valid Values	Description
coord	normalized (default) pixel	Specifies whether the texture coordinates when sampling from a texture are normalized or unnormalized values.
address	repeat mirrored_repeat clamp_to_edge (default) clamp_to_zero clamp_to_border	Sets the addressing mode for all texture coordinates.
s_address t_address r_address	repeat mirrored_repeat clamp_to_edge (default) clamp_to_zero clamp_to_border	Sets the addressing mode for individual texture coordinates.
filter	nearest (default) linear bicubic	Sets the magnification and minification filtering modes for texture sampling.
mag_filter	nearest (default) linear bicubic	Sets the magnification filtering mode for texture sampling.
min_filter	nearest (default) linear bicubic	Sets the minification filtering mode for texture sampling.
mip_filter	none (default) nearest linear	Sets the mipmap filtering mode for texture sampling. If none, then only one level-of-detail is active.
compare_func	never (default) less less_equal greater greater_equal equal not_equal always	Sets comparison test that will be used by the <code>sample_compare</code> and <code>gather_compare</code> texture functions.

The enumeration types used by the sampler data type as described in Table 2 are specified as follows:

```
enum class coord1 { normalized, pixel };
enum class filter { nearest, linear };
```

¹ If coord is set to pixel, the min_filter and mag_filter values must be the same, the mip_filter value must be none, and the address modes must be either clamp_to_zero, clamp_to_border or clamp_to_edge.

```

enum class min_filter { nearest, linear };
enum class mag_filter { nearest, linear };
enum class s_address { clamp_to_zero, clamp_to_edge,
                      repeat, mirrored_repeat };
enum class t_address { clamp_to_zero, clamp_to_edge,
                      repeat, mirrored_repeat };
enum class r_address { clamp_to_zero, clamp_to_edge,
                      repeat, mirrored_repeat };
enum class address { clamp_to_zero, clamp_to_edge,
                    repeat, mirrored_repeat };
enum class mip_filter { none, nearest, linear };

enum class compare_func { none, less, less_equal, greater,
                        greater_equal, equal, not_equal };

enum class border_color2 { transparent_black, opaque_black,
                          opaque_white };

```

In addition to the enumeration types, the following types can also be specified with a sampler:

```

max_anisotropy(int value)
lod_clamp(float min, float max)

```

Metal implements the sampler objects as follows:

```

struct sampler {
    public:
        // full version of sampler constructor
        template<typename... Ts>
        constexpr sampler(Ts... sampler_params){};
    private:
};

```

Ts must be the enumeration types listed above that can be used by the sampler data type. If the same enumeration type is declared multiple times in a given sampler constructor, the last listed value will take effect.

The following Metal program source illustrates several ways to declare samplers. (The attribute qualifiers (`sampler(n)`, `buffer(n)`, and `texture(n)`) that appear in the code below are explained in section

² `border_color` is only valid if the address mode is `clamp_to_border`.

4.3.1.). Note that samplers or constant buffers declared in program source do not need these attribute qualifiers.

```
constexpr sampler s(coord::pixel,
                    address::clamp_to_zero,
                    filter::linear);

constexpr sampler a(coord::normalized);

constexpr sampler b(address::repeat);

constexpr sampler s(address::clamp_to_zero,
                    filter::linear,
                    compare_func::less);

constexpr sampler s(address::clamp_to_zero,
                    filter::linear,
                    compare_func::less,
                    max_anisotropy(10),
                    lod_clamp(0.0f, MAXFLOAT));

kernel void
my_kernel(device float4 *p [[ buffer(0) ]],
          texture2d<float> img [[ texture(0) ]],
          sampler smp [[ sampler(3) ]],
          ...)
{
    ...
}
```

NOTE: Samplers that are initialized in the Metal program source must be declared with the **constexpr** qualifier.

2.7 Arrays and Structs

Arrays and structs are supported with the following restrictions:

- Arrays of `sampler` types are not supported.
- The `texture` and `sampler` types can only be declared in a struct if it is passed by value to a graphics or kernel function.

- The `array<T, N>` type used to declare an array of textures cannot be declared in a struct.
- Arguments to graphics and kernel functions cannot be declared to be of type `size_t`, `ptrdiff_t` or a struct and/or union that contain members declared to be one of these built-in scalar types.
- Members of a struct must belong to the same address space.

2.7.1 Arrays of Textures

An array of textures is declared as:

```
array<typename T, size_t N> or
const array<typename T, size_t N>
```

T must be a texture type described in section 2.6 declared with the `access::read` or `access::sample` qualifier.

Arrays of textures can be passed as an argument to functions (graphics, kernel or user functions) or be declared as local variables inside functions. The following member functions are available for the array of texture type:

2.7.1.1 Element Access

Elements of an array of textures can be accessed using the `[]` operator. The following variants of the `[]` operator are available:

```
reference operator[] (size_t pos) const;
constexpr const_reference operator[] (size_t pos) const;
```

2.7.1.2 Capacity

```
constexpr size_t size();
constexpr size_t size() const;
```

Returns the number of elements in the array.

Examples:

```
#include <metal_stdlib>
using namespace metal;

kernel void
my_kernel(
    const array<texture2d<float>, 10> src [[ texture(0) ]],
    texture2d<float, access::write> dst [[ texture(10) ]],
    ... )
{
    for (int i=0; i<src.size(); i++)
    {
        if (is_null_texture(src[i]))
```

```

        break;
    process_image(src[i], dst);
}
}

```

The Metal shading language also adds support for `array_ref<T3>`. An `array_ref<T>` represents an immutable array of `size()` elements of type `T`. The storage for the array is not owned by the `array_ref<T>` object. Implicit conversion operations are provided from types with contiguous iterators like `metal::array`. A common use for `array_ref<T>` is when passing an array of textures as an argument to functions where you want to be able to accept a variety of array types.

The `array_ref<T>` type cannot be passed as an argument to graphics and kernel functions. The `array_ref<T>` type can, however, be passed as an argument to user functions. The `array_ref<T>` type cannot be declared as local variables inside functions. The following member functions are available for the `array_ref<T>` type:

2.71.3 Element Access

Elements of an `array_ref<T>` can be accessed using the `[]` operator. The following variants of the `[]` operator are available:

```
constexpr const_reference operator[] (size_t pos) const;
```

2.71.4 Capacity

```
constexpr size_t size();
constexpr size_t size() const;
```

Returns the number of elements in the `array_ref<T>`.

2.71.5 Constructors

```
constexpr array_ref();
constexpr array_ref(const array_ref &);
array_ref & operator=(const array_ref &);
constexpr array_ref(const T * array, size_t length);

template<size_t N>
constexpr array_ref(const T(&a) [N]);

template<typename T>
constexpr array_ref<T> make_array_ref(const T * array, size_t length)

template<typename T, size_t N>
constexpr array_ref<T> make_array_ref(const T(&a) [N])

```

Examples:

³ `T` must be one of the supported texture types.

```

#include <metal_stdlib>
using namespace metal;

float4
foo(array_ref<texture2d<float>> src)
{
    float4 clr(0.0f);
    for (int i=0; i<src.size; i++)
    {
        clr += process_texture(src[i]);
    }
    return clr;
}

kernel void
my_kernel_A(
    const array<texture2d<float>, 10> src [[ texture(0) ]],
    texture2d<float, access::write> dst [[ texture(10) ]],
    ... )
{
    float4 clr = foo(src);
    ...
}

kernel void
my_kernel_B(
    const array<texture2d<float>, 20> src [[ texture(0) ]],
    texture2d<float, access::write> dst [[ texture(10) ]],
    ... )
{
    float4 clr = foo(src);
    ...
}

```

2.8 Alignment of Types

Table 3 lists the alignment and size of the scalar and vector data types.

Table 3 Alignment and Size of Scalar and Vector Data Types

Type	Alignment (in bytes)	Size (in bytes)
bool	1	1
char, uchar	1	1
short, ushort	2	2
int, uint	4	4
half	2	2
float	4	4
bool2	2	2
bool3	4	4
bool4	4	4
char2, uchar2	2	2
char3, uchar3	4	4
char4, uchar4	4	4
short2, ushort2	4	4
short3, ushort3	8	8
short4, ushort4	8	8
int2, uint2	8	8
int3, uint3	16	16
int4, uint4	16	16
half2	4	4
half3	8	8
half4	8	8
float2	8	8

Type	Alignment (in bytes)	Size (in bytes)
float3	16	16
float4	16	16

Table 4 lists the alignment and size of the matrix data types.

Table 4 Alignment and Size of Matrix Data Types

Type	Alignment (in bytes)	Size (in bytes)
half2x2	4	8
half2x3	8	16
half2x4	8	16
half3x2	4	12
half3x3	8	24
half3x4	8	24
half4x2	4	16
half4x3	8	32
half4x4	8	32
float2x2	8	16
float2x3	16	32
float2x4	16	32
float3x2	8	24
float3x3	16	48
float3x4	16	48
float4x2	8	32
float4x3	16	64
float4x4	16	64

The `alignas` alignment specifier can be used to specify the alignment requirement of a type or an object. The `alignas` specifier may be applied to the declaration of a variable or a data member of a struct or class. It may also be applied to the declaration of a struct, class or enumeration type.

The Metal compiler is responsible for aligning data items to the appropriate alignment as required by the data type. For arguments to a graphics or kernel function declared to be a pointer to a data type, the

Metal compiler can assume that the pointee is always appropriately aligned as required by the data type.

2.9 Packed Vector Data Types

The vector data types described in section 2.2 are aligned to the size of the vector. There are a number of use cases where developers require their vector data to be tightly packed. For example – a vertex struct that may contain position, normal, tangent vectors and texture coordinates tightly packed and passed as a buffer to a vertex function.

The packed vector type names⁴ supported are:

```
packed_charn, packed_shortn, packed_intn,  
packed_ucharn, packed_ushortn, packed_uintn,  
packed_halfn and packed_floatn
```

n is 2, 3, or 4 representing a 2-, 3- or 4- component vector type.

Table 5 lists the alignment of the packed vector data types.

Table 5 Alignment and Size of Packed Vector Data Types

Type	Alignment (in bytes)	Size (in bytes)
packed_char2, packed_uchar2	1	2
packed_char3, packed_uchar3	1	3
packed_char4, packed_uchar4	1	4
packed_short2, packed_ushort2	2	4
packed_short3, packed_ushort3	2	6
packed_short4, packed_ushort4	2	8
packed_int2, packed_uint2	4	8
packed_int3, packed_uint3	4	12
packed_int4, packed_uint4	4	16
packed_half2	2	4
packed_half3	2	6

⁴ The packed_booln vector type names are reserved.

Type	Alignment (in bytes)	Size (in bytes)
packed_half4	2	8
packed_float2	4	8
packed_float3	4	12
packed_float4	4	16

Packed vector data types are typically used as a data storage format. Loads and stores from a packed vector data type to an aligned vector data type and vice-versa, copy constructor and assignment operator are supported. The arithmetic, logical and relational operators are also supported for packed vector data types.

Examples:

```
device float4 *buffer;
device packed_float4 *packed_buffer;

int i;
packed_float4 f ( buffer[i] );
pack_buffer[i] = buffer[i];

// operator to convert from packed_float4 to float4.
buffer[i] = float4( packed_buffer[i] );
```

Components of a packed vector data type can be accessed with an array index. However, components of a packed vector data type cannot be accessed with the `.xyzw` or `.rgba` selection syntax.

Example:

```
packed_float4 f;

f[0] = 1.0f; // OK
f.x = 1.0f;  // Illegal - compilation error
```

2.10 Implicit Type Conversions

Implicit conversions between scalar built-in types (except void) are supported. When an implicit conversion is done, it is not just a re-interpretation of the expression's value but a conversion of that value to an equivalent value in the new type. For example, the integer value 5 is converted to the floating-point value 5.0.

All vector types are considered to have a higher conversion rank than scalar types. Implicit conversions from a vector type to another vector or scalar type are not permitted and a compilation error results.

For example, the following attempt to convert from a 4-component integer vector to a 4-component floating-point vector fails.

```
int4 i;
float4 f = i;    // compile error.
```

Implicit conversions from scalar-to-vector types are supported. The scalar value is replicated in each element of the vector. The scalar may also be subject to the usual arithmetic conversion to the element type used by the vector or matrix.

For example:

```
float4 f = 2.0f; // f = (2.0f, 2.0f, 2.0f, 2.0f)
```

Implicit conversions from scalar-to-matrix types and vector-to-matrix types are not supported and a compilation error results. Implicit conversions from a matrix type to another matrix, vector or scalar type are not permitted and a compilation error results.

Implicit conversions for pointer types follow the rules described in the C++14 Specification.

2.11 Type Conversions and Re-interpreting Data

The `static_cast5` operator is used to convert from a scalar or vector type to another scalar or vector type with no saturation and with a default rounding mode (i.e., when converting to floating-point, round to nearest even; when converting to integer, round toward zero).

Metal adds an `as_type<type-id>` operator to allow any scalar or vector data type (that is not a pointer) to be reinterpreted as another scalar or vector data type of the same size. The bits in the operand are returned directly without modification as the new type. The usual type promotion for function arguments is not performed.

For example, `as_type<float>(0x3f800000)` returns `1.0f`, which is the value of the bit pattern `0x3f800000` if viewed as an IEEE-754 single precision value.

It is an error to use the `as_type<type-id>` operator to reinterpret data to a type of a different number of bytes.

Examples:

```
float f = 1.0f;
// Legal. Contains: 0x3f800000
uint u = as_type<uint>(f);

// Legal. Contains:
// (int4)(0x3f800000, 0x40000000,
//        0x40400000, 0x40800000)
float4 f = float4(1.0f, 2.0f, 3.0f, 4.0f);
```

⁵ If the source type is a scalar or vector boolean, the value `false` is converted to zero and the value `true` is converted to one.

```
int4 i = as_type<int4>(f);

int i;
// Legal.
short2 j = as_type<short2>(i);

half4 f;
// Error. Result and operand have different sizes
float4 g = as_type<float4>(f);

float4 f;
// Legal. g.xyz will have same values as f.xyz.
// g.w is undefined
float3 g = as_type<float3>(f);
```

3 Operators

This chapter lists and describes the Metal operators.

3.1 Scalar and Vector Operators

1. The arithmetic operators, add (+), subtract (-), multiply (*) and divide (/), operate on scalar and vector, integer and floating-point data types. All arithmetic operators return a result of the same built-in type (integer or floating-point) as the type of the operands, after operand type conversion. After conversion, the following cases are valid:
 - The two operands are scalars. In this case, the operation is applied, and the result is a scalar.
 - One operand is a scalar, and the other is a vector. In this case, the scalar is converted to the element type used by the vector operand. The scalar type is then widened to a vector that has the same number of components as the vector operand. The operation is performed component-wise, which results in a same size vector.
 - The two operands are vectors of the same size. In this case, the operation is performed component-wise, which results in a same size vector.

Division on integer types that results in a value that lies outside of the range bounded by the maximum and minimum representable values of the integer type, such as `TYPE_MIN/-1` for signed integer types or division by zero does not cause an exception but results in an unspecified value. Division by zero for floating-point types results in $\pm\infty$ or NaN, as prescribed by the IEEE-754 standard. (For details about numerical accuracy of floating-point operations, see section 7.)

2. The operator modulus (%) operates on scalar and vector integer data types. All arithmetic operators return a result of the same built-in type as the type of the operands, after operand type conversion. The following cases are valid:
 - The two operands are scalars. In this case, the operation is applied, and the result is a scalar.
 - One operand is a scalar, and the other is a vector. In this case, the scalar is converted to the element type used by the vector operand. The scalar type is then widened to a vector that has the same number of components as the vector operand. The operation is performed component-wise, which results in a same size vector.
 - The two operands are vectors of the same size. In this case, the operation is performed component-wise, which results in a same size vector.

The resulting value is undefined for any component computed with a second operand that is zero, while results for other components with non-zero operands remain defined. If both operands are non-negative, the remainder is non-negative. If one or both operands are negative, results are undefined.

3. The arithmetic unary operators (+ and -) operate on scalar and vector, integer and floating-point types.
4. The arithmetic post- and pre-increment and decrement operators (-- and ++) operate on scalar and vector integer types. All unary operators work component-wise on their operands. The result is the same type they operated on. For post- and pre-increment and decrement, the expression must be one that could be assigned to (an l-value). Pre-increment and pre-decrement add or subtract 1 to the contents of the expression they operate on, and the value of the pre-increment or pre-decrement expression is the resulting value of that modification. Post-

increment and post-decrement expressions add or subtract 1 to the contents of the expression they operate on, but the resulting expression has the expression's value before the post-increment or post-decrement was executed.

5. The relational operators⁶ greater than (`>`), less than (`<`), greater than or equal (`>=`), and less than or equal (`<=`) operate on scalar and vector, integer and floating-point types. The result is a Boolean (`bool` type) scalar or vector. After operand type conversion, the following cases are valid:
 - The two operands are scalars. In this case, the operation is applied, resulting in a `bool`.
 - One operand is a scalar, and the other is a vector. In this case, the scalar is converted to the element type used by the vector operand. The scalar type is then widened to a vector that has the same number of components as the vector operand. The operation is performed component-wise, which results in a Boolean vector.
 - The two operands are vectors of the same type. In this case, the operation is performed component-wise, which results in a Boolean vector.

The relational operators always return `false` if either argument is a NaN.

6. The equality operators, equal (`==`) and not equal (`!=`), operate on scalar and vector, integer and floating-point types. All equality operators result in a Boolean (`bool` type) scalar or vector. After operand type conversion, the following cases are valid:
 - The two operands are scalars. In this case, the operation is applied, resulting in a `bool`.
 - One operand is a scalar, and the other is a vector. In this case, the scalar is converted to the element type used by the vector operand. The scalar type is then widened to a vector that has the same number of components as the vector operand. The operation is performed component-wise, resulting in a Boolean vector.
 - The two operands are vectors of the same type. In this case, the operation is performed component-wise resulting in a same size Boolean vector.

All other cases of implicit conversions are illegal. If one or both arguments is “Not a Number” (NaN), the equality operator equal (`==`) returns `false`. If one or both arguments is “Not a Number” (NaN), the equality operator not equal (`!=`) returns `true`.

7. The bitwise operators and (`&`), or (`|`), exclusive or (`^`), not (`~`) operate on all scalar and vector built-in types except the built-in scalar and vector floating-point types. For built-in vector types, the operators are applied component-wise. If one operand is a scalar and the other is a vector, the scalar is converted to the element type used by the vector operand. The scalar type is then widened to a vector that has the same number of components as the vector operand. The operation is performed component-wise resulting in a same size vector.
8. The logical operators and (`&&`), or (`||`) operate on two Boolean expressions. The result is a scalar or vector Boolean.
9. The logical unary operator not (`!`) operates on a Boolean expression. The result is a scalar or vector Boolean.
10. The ternary selection operator (`?:`) operates on three expressions (`exp1 ? exp2 : exp3`). This operator evaluates the first expression `exp1`, which must result in a scalar Boolean. If the result is

⁶ To test whether any or all elements in the result of a vector relational operator test true. For example, to use in the context of an `if (...)` statement, see the `any` and `all` built-in functions defined in section 5.4.

true, it selects to evaluate the second expression; otherwise it evaluates the third expression. Only one of the second and third expressions is evaluated. The second and third expressions can be any type, as long their types match, or there is a conversion in section 2.10 that can be applied to one of the expressions to make their types match, or one is a vector and the other is a scalar in which case the scalar is widened to the same type as the vector type. This resulting matching type is the type of the entire expression.

11. The ones' complement operator (~). The operand must be of a scalar or vector integer type, and the result is the ones' complement of its operand.

The operators right-shift (>>), left-shift (<<) operate on all scalar and vector integer types. For built-in vector types, the operators are applied component-wise. For the right-shift (>>), left-shift (<<) operators, if the first operand is a scalar, the rightmost operand must be a scalar. If the first operand is a vector, the rightmost operand can be a vector or scalar.

The result of $E1 \ll E2$ is $E1$ left-shifted by $\log_2(N)$ least significant bits in $E2$ viewed as an unsigned integer value, where N is the number of bits used to represent the data type of $E1$, if $E1$ is a scalar, or the number of bits used to represent the type of $E1$ elements, if $E1$ is a vector. The vacated bits are filled with zeros.

The result of $E1 \gg E2$ is $E1$ right-shifted by $\log_2(N)$ least significant bits in $E2$ viewed as an unsigned integer value, where N is the number of bits used to represent the data type of $E1$, if $E1$ is a scalar, or the number of bits used to represent the type of $E1$ elements, if $E1$ is a vector. If $E1$ has an unsigned type or if $E1$ has a signed type and a nonnegative value, the vacated bits are filled with zeros. If $E1$ has a signed type and a negative value, the vacated bits are filled with ones.

12. The assignment operator behaves as described by the C++14 Specification. For the `lvalue = expression` assignment operation, if expression is a scalar type and lvalue is a vector type, the scalar is converted to the element type used by the vector operand. The scalar type is then widened to a vector that has the same number of components as the vector operand. The operation is performed component-wise, which results in a same size vector.

NOTE:

- Operators not described above that are supported by C++14 (such as `sizeof(T)`, unary (&) operator, and comma (,) operator) behave as described in the C++14 Specification.
- Unsigned integers shall obey the laws of arithmetic modulo 2^n , where n is the number of bits in the value representation of that particular size of integer. The result of signed integer overflow is undefined.
- For integral operands the divide (/) operator yields the algebraic quotient with any fractional part discarded⁷; if the quotient a/b is representable in the type of the result, $(a/b)*b + a\%b$ is equal to a .

3.2 Matrix Operators

The arithmetic operators add (+), subtract (-) operate on matrices. Both matrices must have the same numbers of rows and columns. The operation is done component-wise resulting in the same size matrix. The arithmetic operator multiply (*), operates on:

- a scalar and a matrix,

⁷ This is often called truncation towards zero.

- a matrix and a scalar,
- a vector and a matrix,
- a matrix and a vector,
- or a matrix and a matrix.

If one operand is a scalar, the scalar value is multiplied to each component of the matrix resulting in the same size matrix. A right vector operand is treated as a column vector and a left vector operand as a row vector. For vector – matrix, matrix – vector and matrix – matrix multiplication, the number of columns of the left operand is required to be equal to the number of rows of the right operand. The multiply operation does a linear algebraic multiply, yielding a vector or a matrix that has the same number of rows as the left operand and the same number of columns as the right operand.

The examples below presume these vector, matrix, and scalar variables are initialized:

```
float3 v;
float3x3 m;
float a = 3.0f;
```

The following matrix-to-scalar multiplication

```
float3x3 m1 = m * a;
```

is equivalent to:

```
m1[0][0] = m[0][0] * a;
m1[0][1] = m[0][1] * a;
m1[0][2] = m[0][2] * a;
m1[1][0] = m[1][0] * a;
m1[1][1] = m[1][1] * a;
m1[1][2] = m[1][2] * a;
m1[2][0] = m[2][0] * a;
m1[2][1] = m[2][1] * a;
m1[2][2] = m[2][2] * a;
```

The following vector-to-matrix multiplication

```
float3 u = v * m;
```

is equivalent to:

```
u.x = dot(v, m[0]);
u.y = dot(v, m[1]);
u.z = dot(v, m[2]);
```

The following matrix-to-vector multiplication

```
float3 u = m * v;
```

is equivalent to:

```
u = v.x * m[0];  
u += v.y * m[1];  
u += v.z * m[2];
```

The following matrix-to-matrix multiplication

```
float3x3 m, n, r;  
r = m * n;
```

is equivalent to:

```
r[0] = m[0] * n[0].x;  
r[0] += m[1] * n[0].y;  
r[0] += m[2] * n[0].z;  
  
r[1] = m[0] * n[1].x;  
r[1] += m[1] * n[1].y;  
r[1] += m[2] * n[1].z;  
  
r[2] = m[0] * n[2].x;  
r[2] += m[1] * n[2].y;  
r[2] += m[2] * n[2].z;
```

NOTE: The order of partial sums for the vector-to-matrix, matrix-to-vector and matrix-to-matrix multiplication operations described above is undefined.

4 Functions, Variables, and Qualifiers

This chapter describes how functions, arguments, and variables are declared. It also details how qualifiers are often used with functions, arguments, and variables to specify restrictions.

4.1 Function Qualifiers

Metal supports the following qualifiers that restrict how a function may be used:

- `kernel` - A data-parallel function that is executed over a 1-, 2- or 3-dimensional grid.
- `vertex` - A vertex function that is executed for each vertex in the vertex stream and generates per-vertex output or a post-tessellation vertex function that is executed for each surface sample on the patch produced by the fixed-function tessellator.
- `fragment` - A fragment function that is executed for each fragment in the fragment stream and their associated data and generates per-fragment output.

A function qualifier is used at the start of a function, before its return type. The following example shows the syntax for a compute function.

```
kernel void
foo(...)
{
    ...
}
```

For functions declared with the `kernel` qualifier, the return type must be `void`.

Only a graphics function can be declared with one of the `vertex` or `fragment` qualifiers. For graphics functions, the return type identifies whether the output generated by the function is either per-vertex or per-fragment. The return type for a graphics function may be `void` indicating that the function does not generate output.

Functions that use a `kernel`, `vertex` or `fragment` function qualifier cannot call functions that also use these qualifiers, or a compilation error results.

Functions that use a `kernel`, `vertex` or `fragment` function qualifier can be declared within a namespace.

4.1.1 Post-Tessellation Vertex Function

The post-tessellation vertex function calculates the vertex data for each surface sample on the patch produced by the fixed-function tessellator. The inputs to the post-tessellation vertex function are:

- the per-patch data,
- the patch control point data, and
- the tessellator stage output (the normalized vertex location on the patch).

The post-tessellation vertex function generates the final vertex data for the tessellated triangles. For example, to add additional detail (such as displacement mapping values) to the rendered geometry, the

post-tessellation vertex function can sample a texture to modify the vertex position by a displacement value.

After the post-tessellation vertex function has executed, the tessellated primitives are rasterized.

The post-tessellation vertex function is a vertex function identified using the ordinary `vertex` function qualifier. The following attributes must be specified with the post-tessellation vertex function:

4.1.1.1 Patch Type and Number of Control Points Per-Patch

The `[[ patch(patch-type, N) ]]` attribute is used to specify the patch type and optionally⁸ the number of control points in the patch that are associated with the post-tessellation vertex function. `patch-type` must be either `quad` or `triangle`. `N` specifies the number of control-points in the patch and must be a value from 0 to 32.

Example:

```
[[patch(quad)]]
vertex vertex_output
my_post_tessellation_vertex(...)
{
    ...
}
```

```
[[patch(quad, 16)]]
vertex vertex_output
my_bezier_vertex(...)
{
    ...
}
```

4.1.2 Attribute Qualifiers for Fragment Functions

The `[[early_fragment_tests]]` qualifier can be used with a fragment function to request that fragment tests be performed before fragment shader execution.

Below is an example of a fragment function that uses this qualifier:

```
fragment [[ early_fragment_tests ]] float4
my_frag_shader( ... )
{
    ...
}
```

⁸ The optional number of control-points declared with the `[[ patch ]]` attribute is required on macOS. If the number of control points are specified in the post-tessellation vertex function, this number must match the number of control points specified to the `drawPatches` or `drawIndexedPatches` API.

NOTE:

- It is an error if the return type of the fragment shader declared with the `[[early_fragment_tests]]` qualifier includes a depth value i.e. the return type of this fragment function includes an element declared with the `[[depth(depth_qualifier)]]` attribute.
- It is an error to use the `[[early_fragment_tests]]` qualifier with any function that is not a fragment function i.e. not declared with the `fragment` qualifier.

4.2 Address Space Qualifiers for Variables and Arguments

The Metal shading language implements address space qualifiers to specify the region of memory where a function variable or argument is allocated. These qualifiers describe disjoint address spaces for variables:

- `device` (for more details, see section 4.2.1)
- `threadgroup` (see section 4.2.2)
- `constant` (see section 4.2.3)
- `thread` (see section 4.2.4)

All arguments to a graphics or kernel function that are a pointer or reference to a type must be declared with an address space qualifier. For graphics functions, an argument that is a pointer or reference to a type must be declared in the `device` or `constant` address space. For kernel functions, an argument that is a pointer or reference to a type must be declared in the `device`, `threadgroup` or `constant` address space. The following example introduces the use of several address space qualifiers. (The `threadgroup` qualifier is supported here for the pointer `l_data` only if `foo` is called by a kernel function, as detailed in section 4.2.2.)

```
void foo(device int *g_data,
         threadgroup int *l_data,
         constant float *c_data)
{
    ...
}
```

The address space for a variable at program scope must be `constant`.

Any variable that is a pointer or reference must be declared with one of the address space qualifiers discussed in this section. If an address space qualifier is missing on a pointer or reference type declaration, a compilation error occurs.

4.2.1 device Address Space

The `device` address space name refers to buffer memory objects allocated from the device memory pool that are both readable and writeable.

A buffer memory object can be declared as a pointer or reference to a scalar, vector or user-defined struct. The actual size of the buffer memory object is determined when the memory object is allocated via appropriate Metal API calls in the host code.

Some examples are:

```
// an array of a float vector with 4 components
device float4 *color;

struct Foo {
    float a[3];
    int b[2];
}

// an array of Foo elements
device Foo *my_info;
```

Since texture objects are always allocated from the device address space, the `device` address qualifier is not needed for texture types. The elements of a texture object cannot be directly accessed. Functions to read from and write to a texture object are provided.

4.2.2 threadgroup Address Space

The `threadgroup` address space name is used to allocate variables used by a kernel function. Variables declared in the `threadgroup` address space **cannot** be used in graphics functions.

Variables allocated in the `threadgroup` address space in a kernel function are allocated for each threadgroup executing the kernel, are shared by all threads in a threadgroup and exist only for the lifetime of the threadgroup that is executing the kernel.

The example below shows how variables allocated in the `threadgroup` address space can be passed either as arguments or be declared inside a kernel function. (The qualifier `[[ threadgroup(0) ]]` in the code below is explained in section 4.3.1.)

```
kernel void
my_func(threadgroup float *a [[ threadgroup(0) ]], ...)
{
    // A float allocated in the threadgroup address space
    threadgroup float x;

    // An array of 10 floats allocated in the
    // threadgroup address space
    threadgroup float b[10];

    ...
}
```

```
}
```

4.2.3 constant Address Space

The `constant` address space name refers to buffer memory objects allocated from the device memory pool but are read-only. Variables in program scope must be declared in the `constant` address space and initialized during the declaration statement. The initializer(s) expression must be a core constant expression⁹. Variables in program scope have the same lifetime as the program, and their values persist between calls to any of the compute or graphics functions in the program.

```
constant float samples[] = { 1.0f, 2.0f, 3.0f, 4.0f };
```

Pointers or references to the `constant` address space are allowed as arguments to functions.

Writing to variables declared in the `constant` address space is a compile-time error. Declaring such a variable without initialization is also a compile-time error.

NOTE: To decide which address space (**device** or **constant**), a read-only buffer passed to a graphics or kernel function should use, look at how the buffer is accessed inside the graphics or kernel function. The **constant** address space is optimized for multiple instances executing a graphics or kernel function accessing the same location in the buffer. Some examples of this access pattern are accessing light or material properties for lighting / shading, matrix of a matrix array used for skinning, filter weight accessed from a filter weight array for convolution. If multiple executing instances of a graphics or kernel function are accessing the buffer using an index such as the vertex ID, fragment coordinate, or the thread position in grid, the buffer should be allocated in the **device** address space.

4.2.4 thread Address Space

The `thread` address space refers to the per-thread memory address space. Variables allocated in this address space are not visible to other threads. Variables declared inside a graphics or kernel function are allocated in the `thread` address space.

```
kernel void
my_func(...)
{
    // A float allocated in the per-thread address space
    float x;

    // A pointer to variable x in per-thread address space
    thread float p = &x;

    ...
}
```

⁹ Refer to section 5.19 of the C++14 specification.

4.3 Function Arguments and Variables

All inputs¹⁰ and outputs to a graphics and kernel functions are passed as arguments. Arguments to graphics (vertex and fragment) and kernel functions can be one of the following:

- device buffer – a pointer or reference to any data type in the `device` address space (see section 2.4)
- constant buffer – a pointer or reference to any data type in the `constant` address space (see section 2.4)
- `texture` object (see section 2.5)
- `sampler` object (see section 2.6)
- a buffer shared between threads in a threadgroup¹¹ – a pointer to a type in the `threadgroup` address space
- A struct whose elements are either buffers or textures.

Buffers (device and constant) specified as argument values to a graphics or kernel function cannot alias; i.e., a buffer passed as an argument value cannot overlap another buffer passed to a separate argument of the same graphics or kernel function.

The arguments to these functions are often specified with attribute qualifiers to provide further guidance on their use. Attribute qualifiers are used to specify:

- the resource location for the argument (see section 4.3.1),
- built-in variables that support communicating data between fixed-function and programmable pipeline stages (see section 4.3.3),
- which data is sent down the pipeline from vertex function to fragment function (see section 4.3.5).

4.3.1 Attribute Qualifiers to Locate Buffers, Textures and Samplers

For each argument, an attribute qualifier can be optionally specified to identify the location of a buffer, texture or sampler to use for this argument type. The Metal framework API uses this attribute to identify the location for these argument types.

- device and constant buffers – `[[ buffer(index) ]]`
- texture and an array of textures – `[[ texture(index) ]]`
- sampler – `[[ sampler(index) ]]`
- threadgroup buffer – `[[ threadgroup(index) ]]`

The `index` value is an unsigned integer that identifies the location of a buffer, texture or sampler argument that is being assigned. The proper syntax is for the attribute qualifier to follow the argument/variable name.

¹⁰ Except for initialized variables in the constant address space and samplers declared in program scope.

¹¹ Can only be used as arguments with kernel functions.

The example below is a simple kernel function, `add_vectors`, that adds an array of two buffers in the device address space, `inA` and `inB`, and returns the result in the buffer `out`. The attribute qualifiers (`buffer(index)`) specify the buffer locations for the function arguments.

```
kernel void
add_vectors(const device float4 *inA [[ buffer(0) ]],
            const device float4 *inB [[ buffer(1) ]],
            device float4 *out [[ buffer(2) ]],
            uint id [[ thread_position_in_grid ]])
{
    out[id] = inA[id] + inB[id];
}
```

The example below shows attribute qualifiers used for function arguments of several different types (a buffer, a texture, and a sampler):

```
kernel void
my_kernel(device float4 *p [[ buffer(0) ]],
          texture2d<float> img [[ texture(0) ]],
          sampler sam [[ sampler(1) ]])
{
    ...
}
```

If the location indices are not specified the Metal compiler will assign them using the first available location index. In the following example:

```
kernel void
my_kernel(texture2d<half> src,
          texture2d<half, access::write> dst,
          sampler s,
          device myUserInfo *u)
{
    ...
}
```

`src` is assigned texture index 0, `dst` texture index 1, `s` sampler index 0 and `u` buffer index 0.

Consider the following example where the kernel has arguments some of which have location indices assigned and some do not.

```
kernel void
my_kernel(texture2d<half> src [[ texture(0) ]],
```

```

        texture2d<half, access::write> dst,
        sampler s,
        device myUserInfo *u,
        device float *f [[ buffer(10) ]])
    {
        ...
    }

```

src is assigned texture index 0, dst texture index 1, s sampler index 0, u buffer index 0 and f buffer index 10.

NOTE: For buffers (device, constant and threadgroup), textures or samplers the **index** value that is used to specify the buffer, texture or sampler index must be unique. Multiple buffer, texture or sampler arguments with the same **index** value will be a compilation error unless they are declared with a function constant attribute (refer to section 4.9).

4.3.1.1 Vertex function example that specifies resources and outputs to device memory

The following example is a vertex function, `render_vertex`, which outputs to device memory in the array `xform_pos_output`, which is a function argument specified with the `device` qualifier (which was introduced in section 4.2.1). All the `render_vertex` function arguments are specified with qualifiers (`buffer(0)`, `buffer(1)`, `buffer(2)`, and `buffer(3)`), as introduced in section 4.3.1. (The `position` qualifier shown in this example is discussed in section 4.3.3)

```

#include <metal_stdlib>
using namespace metal;

struct VertexOutput {
    float4 position [[position]];
    float4 color;
    float2 texcoord;
};

struct VertexInput {
    float4 position;
    float3 normal;
    float2 texcoord;
};

constexpr constant uint MAX_LIGHTS = 4;

struct LightDesc {

```

```

        uint    num_lights;
        float4  light_position[MAX_LIGHTS];
        float4  light_color[MAX_LIGHTS];
        float4  light_attenuation_factors[MAX_LIGHTS];
    };

    vertex void
    render_vertex(const device VertexInput* v_in [[ buffer(0) ]],
                  constant float4x4& mvp_matrix [[ buffer(1) ]],
                  constant LightDesc& light_desc [[ buffer(2) ]],
                  device VertexOutput* xform_output [[ buffer(3) ]],
                  uint v_id [[ vertex_id ]] )
    {
        VertexOutput v_out;
        v_out.position = v_in[v_id].position * mvp_matrix;
        v_out.color = do_lighting(v_in[v_id].position,
                                  v_in[v_id].normal,
                                  light_desc);

        v_out.texcoord = v_in[v_id].texcoord;

        // output position to a buffer
        xform_pos_output[v_id] = v_out;
    }

```

4.3.2 Struct of buffers and textures

Arguments to a graphics, kernel or user function can be a struct or a nested struct whose members are buffers, textures or samplers only. Such a struct must be passed by value. Each member of such a struct passed as the argument type to a graphics or kernel function can have an attribute qualifier to specify its location (as described in section 4.3.1).

Examples:

```

struct Foo {
    texture2d<float>  a [[ texture(0) ]];
    depth2d<float>    b [[ texture(1) ]];
};

kernel void
my_kernel(Foo f)
{

```

```

    ...
}

```

Below are some examples of invalid use cases which should result in a compilation error.

kernel void

```

my_kernel(device Foo& f) // illegal use
{
    ...
}

struct MyResources {
    texture2d<float>  a [[ texture(0) ]];
    depth2d<float>    b [[ texture(1) ]];
    int c;
};

kernel void
my_kernel(MyResources r) // illegal use
{
}

```

Nested structs are also supported as shown by the following example.

```

struct Foo {
    texture2d<float>  a [[ texture(0) ]];
    depth2d<float>    b [[ texture(1) ]];
};

struct Bar {
    Foo f;
    sampler s [[ sampler(0) ]];
};

kernel void
my_kernel(Bar b)
{
    ...
}

```

4.3.3 Attribute Qualifiers to Locate Per-Vertex Inputs

A vertex function can read per-vertex inputs by indexing into a buffer(s) passed as arguments to the vertex function using the vertex and instance IDs. In addition, per-vertex inputs can also be passed as an argument to a vertex function by declaring them with the `[[stage_in]]` attribute qualifier. For per-vertex inputs passed as an argument declared with the `stage_in` qualifier, each element of the per-vertex input must specify the vertex attribute location as

```
[[ attribute(index) ]]
```

The `index` value is an unsigned integer that identifies the vertex input location that is being assigned. The proper syntax is for the attribute qualifier to follow the argument/variable name. The Metal API uses this attribute to identify the location of the vertex buffer and describe the vertex data such as the buffer to fetch the per-vertex data from, its data format, and stride.

The example below shows how vertex attributes can be assigned to elements of a vertex input struct passed to a vertex function using the `stage_in` qualifier.

```
#include <metal_stdlib>
using namespace metal;

struct VertexInput {
    float4 position [[ attribute(0) ]];
    float3 normal    [[ attribute(1) ]];
    half4 color      [[ attribute(2) ]];
    half2 texcoord   [[ attribute(3) ]];
};

constexpr constant uint MAX_LIGHTS = 4;

struct LightDesc {
    uint    num_lights;
    float4  light_position[MAX_LIGHTS];
    float4  light_color[MAX_LIGHTS];
    float4  light_attenuation_factors[MAX_LIGHTS];
};

constexpr sampler s = sampler(coord::normalized,
                               address::clamp_to_zero,
                               filter::linear);

vertex VertexOutput
render_vertex(VertexInput v_in [[ stage_in ]],
              constant float4x4& mvp_matrix [[ buffer(1) ]],
```

```

        constant LightDesc& lights [[ buffer(2) ]],
        uint v_id [[ vertex_id ]])
{
    VertexOutput v_out;
    ...
    return v_out;
}

```

The example below shows how both buffers and the `stage_in` qualifier can be used to fetch per-vertex inputs in a vertex function.

```

#include <metal_stdlib>
using namespace metal;

struct VertexInput {
    float4 position [[ attribute(0) ]];
    float3 normal    [[ attribute(1) ]];
};

struct VertexInput2 {
    half4 color;
    half2 texcoord[4];
};

constexpr constant uint MAX_LIGHTS = 4;

struct LightDesc {
    uint    num_lights;
    float4  light_position[MAX_LIGHTS];
    float4  light_color[MAX_LIGHTS];
    float4  light_attenuation_factors[MAX_LIGHTS];
};

constexpr sampler s = sampler(coord::normalized,
                               address::clamp_to_zero,
                               filter::linear);

vertex VertexOutput
render_vertex(VertexInput v_in [[ stage_in ]],

```

```

        VertexInput2 v_in2 [[ buffer(0) ]],
        constant float4x4& mvp_matrix [[ buffer(1) ]],
        constant LightDesc& lights [[ buffer(2) ]],
        uint v_id [[ vertex_id ]])
{
    VertexOutput vOut;
    ...
    return vOut;
}

```

A post-tessellation vertex function can read the per-patch and patch control-point data. The patch control-point data is specified in the post-tessellation vertex function as the following templated type:

```
patch_control_point<T>
```

where T is a user defined struct. Each element of T must specify an attribute location using [[attribute(index)]].

Member functions are supported by the patch_control_point<T> type are:

```
constexpr size_t size() const;
```

Returns the number of control-points in the patch

```
constexpr const_reference operator[] (size_t pos) const;
```

Returns the data for a specific patch control point identified by pos.

Example:

```

struct ControlPoint {
    int3 patchParam [[ attribute(0) ]];
    float3 P [[ attribute(1) ]];
    float3 P1 [[ attribute(2) ]];
    float3 P2 [[ attribute(3) ]];
    float2 vSegments [[ attribute(4) ]];
};

struct PerPatchData {
    float4 patchConstant [[ attribute(5) ]];
    float4 someOtherPatchConstant [[ attribute(6) ]];
};

struct PatchData {
    patch_control_point<ControlPoint> cp; // control-point data
    PerPatchData patchData; // per-patch data
}

```

```
};

[[ patch(quad) ]]
vertex VertexOutput
post_tess_vertex_func(PatchData input [[ stage_in ]],
                      ...)
{
    ...
}
```

4.3.4 Attribute Qualifiers for Built-in Variables

Some graphics operations occur in the fixed-function pipeline stages and need to provide values to or receive values from graphics functions. *Built-in* input and output variables are used to communicate values between the graphics (vertex and fragment) functions and the fixed-function graphics pipeline stages. Attribute qualifiers are used with arguments and the return type of graphics functions to identify these built-in variables.

4.3.4.1 Attribute Qualifiers for Vertex Function Input

Table 6 lists the built-in attribute qualifiers that can be specified for arguments to a vertex function and the corresponding data types with which they can be used.

Table 6 Attribute Qualifiers for Vertex Function Input Arguments

Attribute Qualifier	Corresponding Data Types	Description
[[vertex_id]]	ushort or uint	The per-vertex identifier. The per-vertex identifier includes the base vertex value if one is specified.
[[instance_id]]	ushort or uint	The per-instance identifier. The per-instance identifier includes the base instance value if one is specified.
[[base_vertex]]	ushort or uint	The base vertex value added to each vertex identifier before reading per-vertex data.
[[base_instance]]	ushort or uint	The base instance value added to each instance identifier before reading per-instance data.

Notes on vertex function input attribute qualifiers:

- If the type used to declare `[[vertex_id]]` is `uint`, the type used to declare `[[base_vertex]]` must be `uint` or `ushort`.
- If the type used to declare `[[vertex_id]]` is `ushort`, the type used to declare `[[base_vertex]]` must be `ushort`.
- If the type used to declare `[[instance_id]]` is `uint`, the type used to declare `[[base_instance]]` must be `uint` or `ushort`.
- If the type used to declare `[[instance_id]]` is `ushort`, the type used to declare `[[base_instance]]` must be `ushort`.

4.3.4.2 Attribute Qualifiers for Post-Tessellation Vertex Function Input

Table 7 lists the built-in attribute qualifiers that can be specified for arguments to a post-tessellation vertex function and the corresponding data types with which they can be used.

Table 7 Attribute Qualifiers for Post-Tessellation Vertex Function Input Arguments

Attribute Qualifier	Corresponding Data Types	Description
<code>[[patch_id]]</code>	<code>ushort</code> or <code>uint</code>	The patch identifier.
<code>[[instance_id]]</code>	<code>ushort</code> or <code>uint</code>	The per-instance identifier. The per-instance identifier includes the base instance value if one is specified.
<code>[[base_instance]]</code>	<code>ushort</code> or <code>uint</code>	The base instance value added to each instance identifier before reading per-instance data.
<code>[[position_in_patch]]</code>	<code>float2</code> or <code>float3</code>	Defines the location on the patch being evaluated. For quad patches, must be <code>float2</code> . For triangle patches, must be <code>float3</code> .

Notes on vertex function input attribute qualifiers:

- If the type used to declare `[[instance_id]]` is `uint`, the type used to declare `[[base_instance]]` must be `uint` or `ushort`.
- If the type used to declare `[[instance_id]]` is `ushort`, the type used to declare `[[base_instance]]` must be `ushort`.

4.3.4.3 Attribute Qualifiers for Vertex Function Output

Table 8 lists the built-in attribute qualifiers that can be specified for a return type of a vertex function or the members of a struct that are returned by a vertex function (and the corresponding data types with which they can be used).

Table 8 Attribute Qualifiers for Vertex Function Return Type

Attribute Qualifier	Corresponding Data Types
[[clip_distance]]	float or float[n] n must be known at compile time
[[point_size]]	float
[[position]]	float4
[[render_target_array_index]]	uchar, ushort or uint The render target array index. This refers to the face of a cubemap, an array slice of a texture array, or an array slice, face of a cubemap array. For a cubemap the render target array index is the face index, which is a value from 0 to 5. For a cubemap array the render target array index is computed as: array slice index * 6 + face index.

The example below describes a vertex function called `process_vertex`. The function returns a user-defined struct called `VertexOutput`, which contains a built-in variable that represents the vertex position, so it requires the `[[position]]` qualifier.

```

struct VertexOutput {
    float4 position [[position]];
    float4 color;
    float2 texcoord;
}

vertex VertexOutput
process_vertex(...)
{
    VertexOutput v_out;

    // compute per-vertex output
    ...

    return v_out;
}

```

NOTE: Post-tessellation vertex function outputs are the same as a regular vertex function.

4.3.4.4 Attribute Qualifiers for Fragment Function Input

Table 9 lists the built-in attribute qualifiers that can be specified for arguments of a fragment function (and their corresponding data types).

NOTE: If the return type of a vertex function is not **void**, it must include the vertex position. If the vertex return type is **float4** this always refers to the vertex position (and the `[[ position ]]` qualifier need not be specified). If the vertex return type is a struct, it must include an element declared with the `[[ position ]]` qualifier.

Table 9 Attribute Qualifiers for Fragment Function Input Arguments

Attribute Qualifier	Corresponding Data Types	Description
<code>[[color(m)]]</code>	<code>floatn</code> , <code>halfn</code> , <code>intn</code> , <code>uintn</code> , <code>shortn</code> or <code>ushortn</code> <code>n</code> must be known at compile time	The input value read from a color attachment. The index <code>m</code> indicates which color attachment to read from.
<code>[[front_facing]]</code>	<code>bool</code>	This value is <code>true</code> if the fragment belongs to a front-facing primitive.
<code>[[point_coord]]</code>	<code>float2</code>	Two-dimensional coordinates indicating where within a point primitive the current fragment is located. They range from 0.0 to 1.0 across the point.
<code>[[position]]</code>	<code>float4</code>	Describes the window relative coordinate (<code>x</code> , <code>y</code> , <code>z</code> , <code>1/w</code>) values for the fragment.
<code>[[sample_id]]</code>	<code>uint</code>	The sample number of the sample currently being processed.
<code>[[sample_mask]]</code>	<code>uint</code>	The set of samples covered by the primitive generating the fragment during multisample rasterization.
<code>[[render_target_array_index]]</code>	<code>uchar</code> , <code>ushort</code> or <code>uint</code>	The render target array index. This refers to the face of a cubemap, an array slice of a texture array, or an array slice, face of a cubemap array. For a cubemap the render target array index is the face index, which is a value from 0 to 5. For a cubemap array the render target array index is computed as: array slice index * 6 + face index.

A variable declared with the `[[position]]` attribute as input to a fragment function can only be declared with the `center_no_perspective` sampling and interpolation qualifier.

For `[[color(m)]]`, `m` is used to specify the color attachment index when accessing (reading or writing) multiple color attachments in a fragment function.

4.3.4.5 Attribute Qualifiers for Fragment Function Output

The return type of a fragment function describes the per-fragment output. A fragment function can output one or more render-target color values, a depth value, and a coverage mask, which must be identified by using the attribute qualifiers listed in Table 10. If the depth value is not output by the fragment function, the depth value generated by the rasterizer is output to the depth attachment.

Table 10 Attribute Qualifiers for Fragment Function Return Types

Attribute Qualifier	Corresponding Data Types
[[color(m)]] [[color(m), index(i)]]	floatn, halfn, intn, uintn, shortn or ushortn m is the color attachment index and must be known at compile time. The index i can be used to specify one or more colors output by a fragment function for a given color attachment and is an input to the blend equation. If index(i) is not specified, an index of 0 is assumed. Index i, if specified, must be known at compile time. Multiple elements in the fragment function return type that use the same color attachment index must be declared with the same data type.
[[depth(depth_qualifier)]]	float
[[sample_mask]]	uint

The color attachment index *m* for fragment output is specified in the same way as it is for [[color(m)]] for fragment input (see discussion for Table 9).

If there is only a single color attachment in a fragment function, then [[color(m)]] is optional. If [[color(m)]] is not specified, the attachment index will be 0. If multiple color attachments are specified, [[color(m)]] must be specified for all color values. See examples of specifying the color attachment in sections 4.6 and 4.7.

If a fragment function writes a depth value, the *depth_qualifier* must be specified with one of the following values:

- any
- greater
- less

The following example shows how color attachment indices can be specified. Color values written in *clr_f* write to color attachment index 0, *clr_i* to color attachment index 1, and *clr_ui* to color attachment index 2.

```
struct MyFragmentOutput {
```

```

        // color attachment 0
        float4 clr_f [[color(0)]];

        // color attachment 1
        int4 clr_i [[color(1)]];

        // color attachment 2
        uint4 clr_ui [[color(2)]];
    }

    fragment MyFragmentOutput
    my_frag_shader( ... )
    {
        MyFragmentOutput f;
        ....
        f.clr_f = ...;
        ...
        return f;
    }

```

NOTE: If a color attachment index is used both as an input to and output of a fragment function, the data types associated with the input argument and output declared with this color attachment index must match.

4.3.4.6 Attribute Qualifiers for Kernel Function Input

When a kernel is submitted for execution, it executes over an N-dimensional grid of threads, where N is one, two or three. A thread is an instance of the kernel that executes for each point in this grid, and `thread_position_in_grid` identifies its position in the grid.

Threads are organized into **threadgroups**. Threads in a threadgroup cooperate by sharing data through threadgroup memory and by synchronizing their execution to coordinate memory accesses to both device and threadgroup memory. The threads in a given threadgroup execute concurrently on a single compute unit¹² on the GPU. Within a compute unit, a threadgroup is partitioned into multiple smaller groups for execution. The execution width of the compute unit, referred to as the `thread_execution_width`, determines the recommended size of this smaller group. For best performance, the total number of threads in the threadgroup should be a multiple of the `thread_execution_width`.

¹² A GPU may have multiple compute units. Multiple threadgroups can execute concurrently across multiple compute units.

Threadgroups are assigned a unique position within the grid (referred to as `threadgroup_position_in_grid`). Threads are assigned a unique position within a threadgroup (referred to as `thread_position_in_threadgroup`). The unique scalar index of a thread within a threadgroup is given by `thread_index_in_threadgroup`.

Each thread's position in the grid and position in the threadgroup are N-dimensional tuples. Threadgroups are assigned a position using a similar approach to that used for threads. Threads are assigned to a threadgroup and given a position in the threadgroup with components in the range from zero to the size of the threadgroup in that dimension minus one.

When a kernel is submitted for execution, the number of threadgroups and the threadgroup size are specified or the number of threads in the grid and the threadgroup size are specified. For example, consider a kernel submitted for execution that uses a 2-dimensional grid where the number of threadgroups specified are (W_x, W_y) and the threadgroup size is (S_x, S_y) . Let (w_x, w_y) be the position of each threadgroup in the grid (i.e., `threadgroup_position_in_grid`) and (l_x, l_y) be the position of each thread in the threadgroup (i.e., `thread_position_in_threadgroup`).

The thread position in the grid (i.e., `thread_position_in_grid`) is:

$$(g_x, g_y) = (w_x * S_x + l_x, w_y * S_y + l_y)$$

The grid size (i.e., `threads_per_grid`) is:

$$(G_x, G_y) = (W_x * S_x, W_y * S_y)$$

The thread index in the threadgroup (i.e., `thread_index_in_threadgroup`) is:

$$l_y * S_x + l_x$$

Table 11 lists the built-in attribute qualifiers that can be specified for arguments to a kernel function and the corresponding data types with which they can be used.

Table 11 Attribute Qualifiers for Kernel Function Input Arguments

Attribute Qualifier	Corresponding Data Types
<code>[[thread_position_in_grid]]</code>	ushort, ushort2, ushort3, uint, uint2 or uint3
<code>[[thread_position_in_threadgroup]]</code>	ushort, ushort2, ushort3, uint, uint2 or uint3
<code>[[thread_index_in_threadgroup]]</code>	ushort or uint
<code>[[threadgroup_position_in_grid]]</code>	ushort, ushort2, ushort3, uint, uint2 or uint3
<code>[[threads_per_grid]]</code>	ushort, ushort2, ushort3, uint, uint2 or uint3
<code>[[threads_per_threadgroup]]</code>	ushort, ushort2, ushort3, uint, uint2 or uint3

Attribute Qualifier	Corresponding Data Types
<code>[[threadgroups_per_grid]]</code>	<code>ushort, ushort2, ushort3, uint, uint2 or uint3</code>
<code>[[thread_execution_width]]</code>	<code>ushort or uint</code>

Notes on kernel function attribute qualifiers:

- The type used to declare `[[thread_position_in_grid]]`, `[[threads_per_grid]]`, `[[thread_position_in_threadgroup]]`, `[[threads_per_threadgroup]]`, `[[threadgroup_position_in_grid]]` and `[[threadgroups_per_grid]]` must be a scalar type or a vector type. If it is a vector type, the number of components for the vector types used to declare these arguments must match.
- The data types used to declare `[[thread_position_in_grid]]` and `[[threads_per_grid]]` must match.
- The data types used to declare `[[thread_position_in_threadgroup]]` and `[[threads_per_threadgroup]]` must match.
- If `[[thread_position_in_threadgroup]]` is declared to be of type `uint`, `uint2` or `uint3`, then `[[thread_index_in_threadgroup]]` must be declared to be of type `uint`.

4.3.5 stage_in Qualifier

The per-fragment inputs to a fragment function are generated using the output from a vertex function and the fragments generated by the rasterizer. The per-fragment inputs are identified using the `[[stage_in]]` attribute qualifier.

A vertex function can read per-vertex inputs by indexing into a buffer(s) passed as arguments to the vertex function using the vertex and instance IDs. In addition, per-vertex inputs can also be passed as arguments to a vertex function by declaring them with the `[[stage_in]]` attribute qualifier.

A kernel function reads per-thread inputs by indexing into a buffer(s) or texture(s) passed as arguments to the kernel function using the thread position in grid or thread position in threadgroup IDs. In addition, per-thread inputs can also be passed as arguments to a kernel function by declaring them with the `[[stage_in]]` attribute qualifier.

Only one argument of the vertex, fragment or kernel function can be declared with the `stage_in` qualifier. For a user-defined struct declared with the `stage_in` qualifier, the members of the struct can be:

- a scalar integer or floating-point value or
- a vector of integer or floating-point values.

NOTE: Packed vectors, matrices, structs, references or pointers to a type, and arrays of scalars, vectors, matrices and bitfields are not supported as members of the struct declared with the `stage_in` qualifier.

4.3.5.1 Vertex function example that uses the stage_in qualifier

The following example shows how to pass per-vertex inputs using the `stage_in` qualifier.

```
#include <metal_stdlib>
using namespace metal;

struct VertexOutput {
    float4 position [[position]];
    float4 color;
    float2 texcoord[4];
};

struct VertexInput {
    float4 position [[ attribute(0) ]];
    float3 normal    [[ attribute(1) ]];
    half4 color      [[ attribute(2) ]];
    half2 texcoord   [[ attribute(3) ]];
};

constexpr constant uint MAX_LIGHTS = 4;

struct LightDesc {
    uint    num_lights;
    float4  light_position[MAX_LIGHTS];
    float4  light_color[MAX_LIGHTS];
    float4  light_attenuation_factors[MAX_LIGHTS];
};

constexpr sampler s = sampler(coord::normalized,
                               address::clamp_to_zero,
                               filter::linear);

vertex VertexOutput
render_vertex(VertexInput v_in [[ stage_in ]],
              constant float4x4& mvp_matrix [[ buffer(1) ]],
              constant LightDesc& lights [[ buffer(2) ]],
              uint v_id [[ vertex_id ]])
{
    VertexOutput v_out;
    v_out.position = v_in.position * mvp_matrix;
```



```

        v_out.color = do_lighting(v_in.position,
                                v_in.normal,
                                lights);

        ...
    return v_out;
}

```

4.3.5.2 Fragment function example that uses the stage_in qualifier

An example in section 4.3.3 previously introduces the `process_vertex` vertex function, which returns a `VertexOutput` struct per vertex. In the following example, the output from `process_vertex` is pipelined to become input for a fragment function called `render_pixel`, so the first argument of the fragment function uses the `[[stage_in]]` qualifier and uses the incoming `VertexOutput` type. (In `render_pixel`, the `imgA` and `imgB` 2D textures call the built-in function `sample`, which is introduced in section 5.10.3).

```

#include <metal_stdlib>
using namespace metal;

struct VertexOutput {
    float4 position [[position]];
    float4 color;
    float2 texcoord;
};

struct VertexInput {
    float4 position;
    float3 normal;
    float2 texcoord;
};

constexpr constant uint MAX_LIGHTS = 4;

struct LightDesc {
    uint    num_lights;
    float4  light_position[MAX_LIGHTS];
    float4  light_color[MAX_LIGHTS];
    float4  light_attenuation_factors[MAX_LIGHTS];
};

```

```

constexpr sampler s = sampler(coord::normalized,
                               address::clamp_to_edge,
                               filter::linear);

vertex VertexOutput
render_vertex(const device VertexInput *v_in [[ buffer(0) ]],
              constant float4x4& mvp_matrix [[ buffer(1) ]],
              constant LightDesc& lights [[ buffer(2) ]],
              uint v_id [[ vertex_id ]])
{
    VertexOutput v_out;
    v_out.position = v_in[v_id].position * mvp_matrix;
    v_out.color = do_lighting(v_in[v_id].position,
                             v_in[v_id].normal,
                             lights);

    v_out.texcoord = v_in[v_id].texcoord;
    return v_out;
}

fragment float4
render_pixel(VertexOutput input [[stage_in]],
             texture2d<float> imgA [[ texture(0) ]],
             texture2d<float> imgB [[ texture(1) ]])
{
    float4 tex_clr0 = imgA.sample(s, input.texcoord);
    float4 tex_clr1 = imgB.sample(s, input.texcoord);

    // compute color
    float4 clr = compute_color(tex_clr0, tex_clr1, ...);
    return clr;
}

```

4.3.5.3 Kernel function example that uses the stage_in qualifier

The following example shows how to pass per-thread inputs using the `stage_in` qualifier. Using the `stage_in` qualifier in a kernel function allows you to decouple the data type used to declare the per-thread inputs in the shader from the actual data type used to store the per-thread inputs.

```

#include <metal_stdlib>
using namespace metal;

struct PerThreadInput {
    float4 a [[ attribute(0) ]];
    float3 b [[ attribute(1) ]];
    half4 c  [[ attribute(2) ]];
    half2 d  [[ attribute(3) ]];
};

kernel void
my_kernel(PerThreadInput thread_input [[ stage_in ]],
          ...
          uint t_id [[ thread_position_in_grid ]])
{
    ...
}

```

4.4 Storage Class Specifiers

Metal supports the `static` and `extern` storage class specifiers. Metal does not support the `thread_local` storage class specifiers.

The `extern` storage-class specifier can only be used for functions and variables declared in program scope or variables declared inside a function. The `static` storage-class specifier is only for device variables declared in program scope (see section 4.2.3) and is not for variables declared inside a graphics or kernel function. In the following example, the `static` specifier is incorrectly used by the variables `b` and `c` declared inside a kernel function.

```

#include <metal_stdlib>
using namespace metal;

extern constant float4 noise_table[256];
static constant float4 color_table[256] = { ... }; // static is okay

extern void my_foo(texture2d<float> img);
extern void my_bar(device float *a);

kernel void my_func(texture2d<float> img [[ texture(0) ]],
                    device float *ptr [[ buffer(0) ]])
{

```

```

extern constant float4 a;
static constant float4 b;    // static is an error.
static float c;              // static is an error.

...
my_foo(img);
...
my_bar(ptr);
...
}

```

4.5 Sampling and Interpolation Qualifiers

Sampling and interpolation qualifiers are used with inputs to fragment functions declared with the `stage_in` qualifier. The qualifier determines what sampling method the fragment function uses and how the interpolation is performed, including whether to use perspective-correct interpolation, linear interpolation, or no interpolation.

The sampling and interpolation qualifier can be specified on any structure member declared with the `stage_in` qualifier. The sampling and interpolation qualifiers supported are:

```

center_perspective (default13)
center_no_perspective
centroid_perspective
centroid_no_perspective
sample_perspective
sample_no_perspective
flat

```

The following example is user-defined struct that specifies how data in certain members are interpolated:

```

struct FragmentInput {
    float4 pos [[center_no_perspective]];
    float4 color [[center_perspective]];
    float2 texcoord;
    int index [[flat]];
    float f [[sample_perspective]];
};

```

¹³ Except for the `[[ position ]]` variable.

For integer types, the only valid interpolation qualifier is `flat`.

The sampling qualifier variants (`sample_perspective` and `sample_no_perspective`) interpolate at a sample location rather than at the pixel center. With one of these qualifiers, the fragment function or code blocks in the fragment function that use these variables execute per-sample rather than per-fragment.

4.6 Per-Fragment Function vs. Per-Sample Function

The fragment function is typically executed per-fragment. The sampling qualifier identifies if any fragment input is to be interpolated at per-sample vs. per-fragment. Similarly, the `[[sample_id]]` attribute is used to identify the current sample index and the `[[color(m)]]` attribute is used to identify the destination fragment color or sample color (for a multisampled color attachment) value. If any of these qualifiers are used with arguments to a fragment function, the fragment function may execute per-sample¹⁴ instead of per-pixel.

Only the inputs with sample specified (or declared with the `[[sample_id]]` or `[[color(m)]]` qualifier) differ between invocations per-fragment or per-sample, whereas other inputs still interpolate at the pixel center.

The following example uses the `[[color(m)]]` attribute to specify that this fragment function should be executed on a per-sample basis.

```
#include <metal_stdlib>
using namespace metal;

fragment float4
my_frag_shader(float2 tex_coord [[ stage_in ]],
               texture2d<float> img [[ texture(0) ]],
               sampler s [[ sampler(0) ]],
               float4 framebuffer [[color(0)]])
{
    return c = mix(img.sample(s, tex_coord),
                  framebuffer,
                  mix_factor);
}
```

4.7 Programmable Blending

The fragment function can be used to perform per-fragment or per-sample programmable blending. The color attachment index identified by the `[[color(m)]]` attribute qualifier can be specified as an argument to a fragment function.

¹⁴ The implementation may decide to only execute the code that depends on the per-sample values to execute per-sample and the rest of the fragment function may execute per-fragment.

Below is an OpenGL ES programmable blending example that describes how to paint grayscale onto what is below.

The GLSL version is:

```
#extension GL_APPLE_shader_framebuffer_fetch : require

void main()
{
    // RGB to grayscale
    mediump float lum = dot(gl_LastFragData[0].rgb,
                          vec3(0.30,0.59,0.11));
    gl_FragColor = vec4(lum, lum, lum, 1.0);
}
```

The Metal version equivalent is:

```
#include <metal_stdlib>
using namespace metal;

fragment half4
paint_grayscale(half4 dst_color [[color(0)]])
{
    // RGB to grayscale
    half lum = dot(dst_color.rgb,
                  half3(0.30h, 0.59h, 0.11h));
    return half4(lum, lum, lum, 1.0h);
}
```

4.8 Graphics Function – Signature Matching

A graphics function signature is a list of parameters that are either input to or output from a graphics function.

4.8.1 Vertex – Fragment Signature Matching

There are two kinds of data that can be passed between a vertex and fragment function: user-defined and built-in variables.

The per-instance input to a fragment function is declared with the `[[stage_in]]` qualifier. These are output by an associated vertex function.

Built-in variables are declared with one of the attribute qualifiers defined in section 4.3.3. These are either generated by a vertex function (such as `[[position]]`, `[[point_size]]`, `[[clip_distance]]`), are generated by the rasterizer (such as `[[point_coord]]`, `[[front_facing]]`, `[[sample_id]]`, `[[sample_mask]]`) or refer to a framebuffer color value (such as `[[color]]`) passed as an input to the fragment function.

The built-in variable `[[position]]` must always be returned. The other built-in variables (`[[point_size]]`, `[[clip_distance]]`) generated by a vertex function, if needed, must be declared in the return type of the vertex function but cannot be accessed by the fragment function.

Built-in variables generated by the rasterizer or refer to a framebuffer color value may also be declared as arguments of the fragment function with the appropriate attribute qualifier.

The attribute `[[user(name)]]` syntax can also be used to specify an attribute name for any user-defined variables.

A vertex and fragment function are considered to have matching signatures if:

- There is no input argument with the `[[stage_in]]` qualifier declared in the fragment function.
- For a fragment function argument declared with `[[stage_in]]`, each element in the type associated with this argument can be one of the following: a built-in variable generated by the rasterizer, a framebuffer color value passed as input to the fragment function, or a user-generated output from a vertex function. For built-in variables generated by the rasterizer or framebuffer color values, there is no requirement for a matching type to be associated with elements of the vertex return type. For elements that are user-generated outputs, the following rules apply:
- If the attribute name given by `[[user(name)]]` is specified for an element, then this attribute name must match with an element in the return type of the vertex function, and their corresponding data types must also match.
- If the `[[user(name)]]` attribute name is not specified, then the argument name and types must match.

Below is an example of compatible signatures:

```
struct VertexOutput
{
    float4 position [[position]];
    float3 normal;
    float2 texcoord;
};

vertex VertexOutput
my_vertex_shader(...)
{
    VertexOutput v;

    ...
    return v;
}
```

```

fragment float4
my_fragment_shader(VertexOutput f [[stage_in]], ...)
{
    float4 clr;

    ...
    return clr;
}

fragment float4
my_fragment_shader2(VertexOutput f [[stage_in]],
                    bool is_front_face [[front_facing]],
                    ...)
{
    float4 clr;

    ...
    return clr;
}

```

my_vertex_shader and my_fragment_shader or my_vertex_shader and my_fragment_shader2 can be used together to render a primitive.

Below is another example of compatible signatures:

```

struct VertexOutput
{
    float4 position [[position]];
    float3 vertex_normal [[user(normal)]];
    float2 texcoord [[user(texturecoord)]];
};

struct FragInput
{
    float3 frag_normal [[user(normal)]];
    float4 position [[position]];
    float4 framebuffer_color [[color(0)]];
    bool is_front_face [[front_facing]];
}

```



```

};

vertex VertexOutput
my_vertex_shader(...)
{
    VertexOutput v;

    ...
    return v;
}

fragment float4
my_fragment_shader(FragInput f [[stage_in]], ...)
{
    float4 clr;

    ...
    return clr;
}

```

Below is another example of compatible signatures:

```

struct VertexOutput
{
    float4 position [[position]];
    float3 normal;
    float2 texcoord;
};

struct FragInput
{
    float4 position [[position]];
    float2 texcoord;
};

vertex VertexOutput
my_vertex_shader(...)
{

```

```

    VertexOutput v;

    ...
    return v;
}

fragment float4
my_fragment_shader(FragInput f [[stage_in]], ...)
{
    float4 clr;

    ...
    return clr;
}

```

Below is another example of compatible signatures:

```

struct VertexOutput
{
    float4 position [[position]];
    float3 normal;
    float2 texcoord;
};

vertex VertexOutput
my_vertex_shader(...)
{
    VertexOutput v;

    ...
    return v;
}

fragment float4
my_fragment_shader(float4 p [[position]],
                    ...)
{
    float4 clr;
}

```

```

    ...
    return clr;
}

```

Below is an example of incompatible signatures:

```

struct VertexOutput
{
    float4 position [[position]];
    float3 normal;
    float2 texcoord;
};

struct FragInput
{
    float4 position [[position]];
    half3 normal;
};

vertex VertexOutput
my_vertex_shader(...)
{
    VertexOutput v;

    ...
    return v;
}

fragment float4
my_fragment_shader(FragInput f [[stage_in]], ...)
{
    float4 clr;

    ...
    return clr;
}

```

Below is another example of incompatible signatures:

```

struct VertexOutput
{
    float4 position [[position]];
    float3 normal [[user(normal)]];
    float2 texcoord [[user(texturecoord)]];
};

struct FragInput
{
    float3 normal [[user(foo)]];
    float4 position [[position]];
};

vertex VertexOutput
my_vertex_shader(...)
{
    VertexOutput v;

    ...
    return v;
}

fragment float4
my_fragment_shader(FragInput f [[stage_in]], ...)
{
    float4 clr;

    ...
    return clr;
}

```

4.9 Program Scope Function Constants

In previous revisions of Metal, developers have to use pre-processor macros when writing ubershaders. An ubershader is one large shader that's compiled many times with different pre-processor macro defines to enable different features in the shader. The pre-processor macro defines are specified as input to the Metal shading language compiler to generate the specific variant of the ubershader. A major issue with using pre-processor macros for ubershaders with offline compiling is that all specific variants

of the ubershader that a developer may want to use have to be generated which can result in a significant increase in the size of the shader library assets.

The Metal shading language now allows developers to write ubershaders with the same ease of use as using pre-processor macros but moves the generation of the specific variants of the ubershader to when the pipeline state is created. The developer no longer needs to compile offline all variants of the ubershader that they plan to use in their application.

4.9.1 Specifying program scope function constants

The Metal shading language is extended to allow program scope variables to be declared with the following attribute:

```
[[ function_constant(index) ]]
```

Program scope variables declared with the `[[ function_constant(index) ]]` attribute or program scope variables initialized with variables declared with this attribute are referred to as function constants.

These function constants are not initialized in the Metal shader source but instead their values are specified when the render or compute pipeline state is created. `index` is used to specify a location index that can be used to refer to the function constant variable (instead of by its name) in the runtime.

Examples:

```
constant int a [[ function_constant(0) ]];  
constant bool b [[ function_constant(2) ]];
```

Variables in program scope declared in the constant address space can also be initialized using a function constant(s).

Examples:

```
constant int a [[ function_constant(0) ]];  
constant bool b [[ function_constant(2) ]];  
constant bool c = ((a == 1) && b);  
constant int d = (a * 4);
```

The value of function constants `a` and `b` are specified when the render or compute pipeline state is created.

The following built-in function:

```
bool is_function_constant_defined(name)
```

can be used to determine if a function constant is available i.e. has been defined when the render or compute pipeline state is created. `name` must refer to a function constant variable.

`is_function_constant_defined(name)` returns `true` if the function constant variable is defined when the render or compute pipeline state is created and `false` otherwise.

If a function constant variable value is not defined when the render or compute pipeline state is created and if the graphics or kernel function specified with the render or compute pipeline state uses these

function constants directly i.e. not with `is_function_constant_defined(name)`, an error will be generated when the specialized `MTLFunction`¹⁵ is created.

Function constants can be used in Metal:

- To control code paths that get compiled,
- For specifying the optional arguments of a function (graphics, kernel or user functions) and
- For specifying optional elements of a struct that is declared with the `[[ stage_in ]]` qualifier.

4.9.1.1 Using function constants to control code paths to compile

Consider the following shader which uses pre-processor macros for ubershader constants:

```
struct VertexOutput {
    float4 position [[ position ]];
    float4 color;
};

struct VertexInput {
    float4 position [[ attribute(0) ]];
    float4 offset  [[ attribute(1) ]];
    float4 color   [[ attribute(2) ]];
};

vertex VertexOutput
myVertex(VertexInput vIn [[ stage_in ]])
{
    VertexOutput vOut;

    vOut.position = vIn.position;
#ifdef OFFSET_DEFINED
    vOut.position += vIn.offset;
#endif

#ifdef COLOR_DEFINED
    vOut.color = vIn.color;
#else
```

¹⁵ If the function constant variable is used directly and with `is_function_constant_defined(name)`, an error will most likely not be generated when the specialized `MTLFunction` is created and the behavior of using this `MTLFunction` with a pipeline state object is undefined.

```

        vOut.color = float4(0.0f);
    #endif

    return vOut;
}

```

The corresponding shader now written using function constant variables will be:

```

constant bool offset_defined [[ function_constant(0) ]];
constant bool color_defined [[ function_constant(1) ]];

vertex VertexOutput
myVertex(VertexInput vIn [[ stage_in ]])
{
    VertexOutput vOut;

    vOut.position = vIn.position;
    if (offset_defined)
        vOut.position += vIn.offset;

    if (color_defined)
        vOut.color = vIn.color;
    else
        vOut.color = float4(0.0f);

    return vOut;
}

```

Functions constants can only be a scalar or vector type. Using a user-defined type or an array of a scalar or vector type for a function constant will be a compilation error.

4.9.1.2 Using function constants when declaring the arguments of functions

Arguments to a graphics, kernel or user functions can be declared with the `[[ function_constant(name) ]]` attribute qualifier to identify that the argument is optional. `name` refers to a function constant variable. If the value of the function constant variable given by `name` is non-zero or `true` (determined when the render or compute pipeline state is created), the argument is considered to be declared in the function signature. If the value of the function constant variable given by `name` is `0` or `false`, the argument is considered to **not** be declared in the function signature. If `name` refers to a function constant variable that has not been defined (determined when the render or compute pipeline state is created), the behavior will be the same as `is_function_constant_defined(name)` is used and its value is `false`.

Consider the following fragment function that uses pre-processor macros as follows in its function declaration:

```
fragment half4
myFragmentFunction(
    constant GlobalUniformData *globalUniform [[ buffer(0) ]],
    constant RenderUniformData_ModelWithLightmap *renderUniform
        [[ buffer(1) ]],
    constant MaterialUniformData *materialUniform [[ buffer(2) ]],
    texture2d<float> DiffuseTexture [[ texture(0) ]],
    texture2d<float> LightmapTexture [[ texture(1) ]],
    texture2d<float> FogTexture [[ texture(3) ]],
    #ifdef MED_QUALITY
        texture2d<float> LookupTexture [[ texture(4) ]],
    #endif
    #ifdef REALTIME_SHADOW
        texture2d<float> RealtimeShadowMapTexture [[ texture(10) ]],
    #endif
    sampler DiffuseTextureSampler [[ sampler(0) ]],
    sampler LightmapTextureSampler [[ sampler(1) ]],
    sampler FogTextureSampler [[ sampler(3) ]],
    #ifdef MED_QUALITY
        sampler LookupTextureSampler [[ sampler(4) ]],
    #endif
    #ifdef REALTIME_SHADOW
        sampler RealtimeShadowMapTextureSampler [[ sampler(10) ]],
    #endif
    VertexOutput fragIn [[ stage_in ]])
```

The corresponding fragment function now written using function constants will be:

```
constant bool realtime_shadow [[ function_constant(0) ]];
constant bool med_quality [[ function_constant(1) ]];
constant bool med_quality_defined =
    is_function_constant_defined(med_quality);
constant bool realtime_shadow_defined =
    is_function_constant_defined(realtime_shadow);

fragment half4
```



```

myFragmentFunction(
    constant GlobalUniformData *globalUniform [[ buffer(0) ]],
    constant RenderUniformData_ModelWithLightmap *renderUniform
                                   [[ buffer(1) ]],
    constant MaterialUniformData *materialUniform [[ buffer(2) ]],
    texture2d<float> DiffuseTexture [[ texture(0) ]],
    texture2d<float> LightmapTexture [[ texture(1) ]],
    texture2d<float> FogTexture [[ texture(3) ]],
    texture2d<float> LookupTexture [[ texture(4),
                                   function_constant(med_quality_defined) ]],
    texture2d<float> RealtimeShadowMapTexture [[ texture(10),
                                   function_constant(realtime_shadow_defined) ]],
    sampler DiffuseTextureSampler [[ sampler(0) ]],
    sampler LightmapTextureSampler [[ sampler(1) ]],
    sampler FogTextureSampler [[ sampler(3) ]],
    sampler LookupTextureSampler [[ sampler(4),
                                   function_constant(med_quality_defined) ]],
    sampler RealtimeShadowMapTextureSampler [[ sampler(10),
                                   function_constant(realtime_shadow_defined) ]],
    VertexOutput fragIn [[ stage_in ]])

```

Below is another example that shows how to use function constants with arguments to a function:

```

constant bool hasInputBuffer [[ function_constant(0) ]];

kernel void
kernelOptionalBuffer(
    device int *input [[ buffer(0),
                        function_constant(inputBufferDefined) ]],
    device int *output [[ buffer(1) ]],
    uint tid [[ thread_position_in_grid ]])
{
    if (hasInputBuffer)
        output[tid] = inputA[0] * tid;
    else
        output[tid] = tid;
}

```

4.9.1.3 Using function constants for elements of a `[[ stage_in ]]` struct

Elements of a struct declared with the `[[ stage_in ]]` qualifier to a graphics function can also be declared with the `[[ function_constant(name) ]]` attribute qualifier to identify that the element is optional. `name` refers to a function constant variable. If the value of the function constant variable given by `name` is non-zero or `true` (determined when the render or compute pipeline state is created), the element in the struct is considered to be declared in the function signature. If the value of the function constant variable given by `name` is `0` or `false`, the element is considered to **not** be declared in the struct. If `name` refers to a function constant variable that has not been defined (determined when the render or compute pipeline state is created), the behavior will be the same as `is_function_constant_defined(name)` is used and its value is `false`.

Example:

```
constant bool offset_defined [[ function_constant(0) ]];
constant bool color_defined [[ function_constant(1) ]];

struct VertexOutput {
    float4 position [[ position ]];
    float4 color;
};

struct VertexInput {
    float4 position [[ attribute(0) ]];
    float4 offset    [[ attribute(1),
                        function_constant(offset_defined) ]];
    float4 color     [[ attribute(2),
                        function_constant(color_defined)  ]];
};

vertex VertexOutput
myVertex(VertexInput vIn [[ stage_in ]])
{
    VertexOutput vOut;

    vOut.position = vIn.position;
    if (offset_defined)
        vOut.position += vIn.offset;

    if (color_defined)
        vOut.color = vIn.color;
```

```
    else
        vOut.color = float4(0.0f);

    return vOut;
}
```

4.10 Additional Restrictions

- Writes to a texture are disallowed from a fragment shader in Metal on iOS and tvOS.
- Writes to a texture are disallowed from a vertex shader in Metal on iOS and tvOS.
- Writes to a buffer from a vertex shader are not guaranteed to be visible to reads from the associated fragment shader of a given primitive.
- If a vertex shader does writes to a buffer(s), its return type must be `void`.
- The return type of a vertex or fragment function cannot include an element that is a packed vector type, matrix type, a struct type, a reference or a pointer to a type.
- The number of inputs¹⁶ to a fragment function declared with the `stage_in` qualifier can be a maximum of 128 scalars. If an input to a fragment function is a vector then this counts as `n` scalars where `n` is the number of components in the vector.

¹⁶ Does not include the built-in variables declared with one of the following attributes:
[[`color(m)`]], [[`front_facing`]], [[`sample_id`]] and [[`sample_mask`]].

5 Metal Standard Library

This chapter describes the functions supported by the Metal standard library.

5.1 Namespace and Header Files

The Metal standard library functions and enums are declared in the `metal` namespace. In addition to the header files described in the Metal standard library functions, the `<metal_stdlib>` header is available and can access all the functions supported by the Metal standard library.

5.2 Common Functions

The functions in Table 12 are in the Metal standard library and are defined in the header `<metal_common>`. `T` is one of the scalar or vector floating-point types.

Table 12 Common Functions in the Metal Standard Library

Built-in common functions	Description
<code>T clamp(T x, T minval, T maxval)</code>	Returns <code>fmin(fmax(x, minval), maxval)</code> . Results are undefined if <code>minval > maxval</code> .
<code>T mix(T x, T y, T a)</code>	Returns the linear blend of <code>x</code> and <code>y</code> implemented as: $x + (y - x) * a$ <code>a</code> must be a value in the range 0.0 to 1.0. If <code>a</code> is not in the range 0.0 to 1.0, the return values are undefined.
<code>T saturate(T x)</code>	Clamp the specified value within the range of 0.0 to 1.0.
<code>T sign(T x)</code>	Returns 1.0 if <code>x > 0</code> , -0.0 if <code>x = -0.0</code> , +0.0 if <code>x = +0.0</code> , or -1.0 if <code>x < 0</code> . Returns 0.0 if <code>x</code> is a NaN.
<code>T smoothstep(T edge0, T edge1, T x)</code>	Returns 0.0 if <code>x <= edge0</code> and 1.0 if <code>x >= edge1</code> and performs a smooth Hermite interpolation between 0 and 1 when <code>edge0 < x < edge1</code> . This is useful in cases where you want a threshold function with a smooth transition. This is equivalent to: <pre>t = clamp((x - edge0)/(edge1 - edge0), 0, 1); return t * t * (3 - 2 * t);</pre> Results are undefined if <code>edge0 >= edge1</code> or if <code>x</code> , <code>edge0</code> or <code>edge1</code> is a NaN.

Built-in common functions	Description
<code>T step(T edge, T x)</code>	Returns 0.0 if <code>x < edge</code> , otherwise it return 1.0.

For single precision floating-point, Metal also supports a precise and fast variant of the following common functions: `clamp` and `saturate`. The difference between the fast and precise variants is how NaNs are handled. In the fast variant, the behavior of NaNs is undefined whereas the precise variants follow the IEEE 754 rules for NaN handling. The `ffast-math` compiler option (refer to section 6.2) is used to select the appropriate variant when compiling the Metal source. In addition, the `metal::precise` and `metal::fast` nested namespaces are also available and provide developers a way to explicitly select the fast or precise variant of these common functions.

5.3 Integer Functions

The integer functions in Table 13 are in the Metal standard library and are defined in the header `<metal_integer>`. `T` is one of the scalar or vector integer types. `Tu` is the corresponding unsigned scalar or vector integer type.

Table 13 Integer Functions in the Metal Standard Library

Built-in integer functions	Description
<code>T abs(T x)</code>	Returns $ x $.
<code>T_u absdiff(T x, T y)</code>	Returns $ x-y $ without modulo overflow.
<code>T addsat(T x, T y)</code>	Returns $x + y$ and saturates the result.
<code>T clamp(T x, T minval, T maxval)</code>	Returns $\min(\max(x, \text{minval}), \text{maxval})$. Results are undefined if <code>minval > maxval</code> .
<code>T clz(T x)</code>	Returns the number of leading 0-bits in <code>x</code> , starting at the most significant bit position. If <code>x</code> is 0, returns the size in bits of the type of <code>x</code> or component type of <code>x</code> , if <code>x</code> is a vector
<code>T ctz(T x)</code>	Returns the count of trailing 0-bits in <code>x</code> . If <code>x</code> is 0, returns the size in bits of the type of <code>x</code> or component type of <code>x</code> , if <code>x</code> is a vector.

Built-in integer functions	Description
<code>T extract_bits(T x, uint offset, uint bits)</code>	Extract bits <code>[offset, offset+bits-1]</code> from <code>x</code>, returning them in the least significant bits of the result. For unsigned data types, the most significant bits of the result will be set to zero. For signed data types, the most significant bits will be set to the value of bit <code>offset+bits-1</code>. If <code>bits</code> is zero, the result will be zero. The result will be undefined if the sum of <code>offset</code> and <code>bits</code> is greater than the number of bits used to store the operand.
<code>T hadd(T x, T y)</code>	Returns $(x + y) \gg 1$. The intermediate sum does not modulo overflow.
<code>T insert_bits(T base, T insert, uint offset, uint bits)</code>	Returns the insertion of the <code>bits</code> least-significant bits of <code>insert</code> into <code>base</code>. The result will have bits <code>[offset, offset+bits-1]</code> taken from bits <code>[0, bits-1]</code> of <code>insert</code>, and all other bits taken directly from the corresponding bits of <code>base</code>. If <code>bits</code> is zero, the result will be <code>base</code>. The result will be undefined if the sum of <code>offset</code> and <code>bits</code> is greater than the number of bits used to store the operand.
<code>T madhi(T a, T b, T c)</code>	Returns <code>mulhi(a, b) + c</code> .
<code>T madsat(T a, T b, T c)</code>	Returns $a * b + c$ and saturates the result.
<code>T max(T x, T y)</code>	Returns <code>y</code> if $x < y$, otherwise it returns <code>x</code> .
<code>T min(T x, T x)</code>	Returns <code>y</code> if $y < x$, otherwise it returns <code>x</code> .
<code>T mulhi(T x, T y)</code>	Computes $x * y$ and returns the high half of the product of <code>x</code> and <code>y</code> .
<code>T popcount(T x)</code>	Returns the number of non-zero bits in <code>x</code> .
<code>T reverse_bits(T x)</code>	Returns the reversal of the bits of <code>x</code> . The bit numbered <code>n</code> of the result will be taken from bit $(bits - 1) - n$ of <code>x</code> , where <code>bits</code> is the total number of bits used to represent <code>x</code> .
<code>T rhadd(T x, T y)</code>	Returns $(x + y + 1) \gg 1$. The intermediate sum does not modulo overflow.

Built-in integer functions	Description
<code>T rotate(T v, T i)</code>	For each element in <code>v</code> , the bits are shifted left by the number of bits given by the corresponding element in <code>i</code> . Bits shifted off the left side of the element are shifted back in from the right.
<code>T subsat(T x, T y)</code>	Returns <code>x - y</code> and saturates the result.

5.4 Relational Functions

The relational functions in Table 14 are in the Metal standard library and are defined in the header `<metal_relational>`. `T` is one of the scalar or vector floating-point types. `Ti` is one of the scalar or vector integer or boolean types. `Tb` only refers to the scalar or vector boolean types.

Table 14 Relational Functions in the Metal Standard Library

Built-in relational functions	Description
<code>bool all(T_b x)</code>	Returns true only if all components of <code>x</code> are true.
<code>bool any(T_b x)</code>	Returns true only if any component of <code>x</code> are true.
<code>T_b isfinite(T x)</code>	Test for finite value.
<code>T_b isinf(T x)</code>	Test for infinity value (positive or negative).
<code>T_b isnan(T x)</code>	Test for a NaN.
<code>T_b isnormal(T x)</code>	Test for a normal value.
<code>T_b isordered(T x, T y)</code>	Test if arguments are ordered. <code>isordered()</code> takes arguments <code>x</code> and <code>y</code> and returns the result <code>(x == x) && (y == y)</code> .
<code>T_b isunordered(T x, T y)</code>	Test if arguments are unordered. <code>isunordered()</code> takes arguments <code>x</code> and <code>y</code> and returns true if <code>x</code> or <code>y</code> is NaN and false otherwise.
<code>T_b not(T_b x)</code>	Returns the component-wise logical complement of <code>x</code> .
<code>T select(T a, T b, T_b c)</code> <code>T_i select(T_i a, T_i b, T_b c)</code>	For each component of a vector type, <code>result[i] = c[i] ? b[i] : a[i]</code> For a scalar type, <code>result = c ? b : a</code>
<code>T_b signbit(T x)</code>	Test for sign bit. Returns true if the sign bit is set for the floating-point value in <code>x</code> and false otherwise.

5.5 Math Functions

The math functions in Table 15 are in the Metal standard library and are defined in the header `<metal_math>`. `T` is one of the scalar or vector floating-point types. `Ti` refers only to the scalar or vector integer types.

Table 15 Math Functions in the Metal Standard Library

Built-in math functions	Description
<code>T acos(T x)</code>	Arc cosine function.
<code>T acosh(T x)</code>	Inverse hyperbolic cosine.
<code>T asin(T x)</code>	Arc sine function.
<code>T asinh(T x)</code>	Inverse hyperbolic sine.
<code>T atan(T y_over_x)</code>	Arc tangent function.
<code>T atan2(T y, T x)</code>	Arc tangent of y over x.
<code>T atanh(T x)</code>	Hyperbolic arc tangent.
<code>T ceil(T x)</code>	Round to integral value using the round to positive infinity rounding mode.
<code>T copysign(T x, T y)</code>	Return x with its sign changed to match the sign of y.
<code>T cos(T x)</code>	Compute cosine.
<code>T cosh(T x)</code>	Compute hyperbolic cosine.
<code>T cospi(T x)</code>	Compute $\cos(\pi x)$.
<code>T exp(T x)</code>	Compute the base- e exponential of x.
<code>T exp2(T x)</code>	Exponential base 2 function.
<code>T exp10(T x)</code>	Exponential base 10 function.
<code>T fabs(T x)</code> <code>T abs(T x)</code>	Compute absolute value of a floating-point number.
<code>T fdim(T x, T y)</code>	$x - y$ if $x > y$, +0 if x is less than or equal to y.
<code>T floor(T x)</code>	Round to integral value using the round to negative infinity rounding mode.
<code>T fma(T a, T b, T c)</code>	Returns the correctly rounded floating-point representation of the sum of c with the infinitely precise product of a and b. Rounding of intermediate products shall not occur. Edge case behavior is per the IEEE 754-2008 standard.

Built-in math functions	Description
<code>T fmax(T x, T y)</code> <code>T max(T x, T y)</code>	Returns y if $x < y$, otherwise it returns x. If one argument is a NaN, fmax() returns the other argument. If both arguments are NaNs, fmax() returns a NaN.
<code>T fmin(T x, T y)</code> <code>T min(T x, T y)</code>	Returns y if $y < x$, otherwise it returns x. If one argument is a NaN, fmin() returns the other argument. If both arguments are NaNs, fmin() returns a NaN.
<code>T fmod(T x, T y)</code>	Returns $x - y * \text{trunc}(x/y)$.
<code>T fract(T x)</code>	Returns the fractional part of x which is greater than or equal to 0 or less than 1.
<code>T frexp(T x, T_i &exponent)</code>	Extract mantissa and exponent from x. For each component the mantissa returned is a float with magnitude in the interval $[1/2, 1)$ or 0. Each component of x equals mantissa returned * 2^{exp} .
<code>T_i ilogb(T x)</code>	Return the exponent as an integer value.
<code>T ldexp(T x, T_i k)</code>	Multiply x by 2 to the power k.
<code>T log(T x)</code>	Compute natural logarithm.
<code>T log2(T x)</code>	Compute a base 2 logarithm.
<code>T log10(T x)</code>	Compute a base 10 logarithm.
<code>T modf(T x, T &intval)</code>	Decompose a floating-point number. The modf function breaks the argument x into integral and fractional parts each of which has the same sign as the argument. Returns the fractional value. The integral value is returned in intval.
<code>T pow(T x, T y)</code>	Compute x to the power y.
<code>T powr(T x, T y)</code>	Compute x to the power y, where x is ≥ 0 .
<code>T rint(T x)</code>	Round to integral value using round to nearest even rounding mode in floating-point format.
<code>T round(T x)</code>	Return the integral value nearest to x rounding halfway cases away from zero.
<code>T rsqrt(T x)</code>	Compute inverse square root.
<code>T sin(T x)</code>	Compute sine.
<code>T sincos(T x, T &cosval)</code>	Compute sine and cosine of x. The computed sine is the return value and compute cosine is returned in cosval.

Built-in math functions	Description
T sinh(T x)	Compute hyperbolic sine.
T sinpi(T x)	Compute $\sin(\pi x)$.
T sqrt(T x)	Compute square root.
T tan(T x)	Compute tangent.
T tanh(T x)	Compute hyperbolic tangent.
T tanpi(T x)	Compute $\tan(\pi x)$.
T trunc(T x)	Round to integral value using the round to zero rounding mode.

For single precision floating-point, Metal supports two variants of the math functions listed in the table above: the precise and the fast variants. The `-ffast-math` compiler option (refer to section 6.2) is used to select the appropriate variant when compiling the Metal source. In addition, the `metal::precise` and `metal::fast` nested namespaces are also available and provide developers a way to explicitly select the fast or precise variant of these math functions for single precision floating-point.

Examples:

```
#include <metal_stdlib>
using namespace metal;

float x;
float a = sin(x);          // use fast or precise version of sin based on
                          // whether -ffast-math is specified as
                          // compile option.
float b = fast::sin(x);    // use fast version of sin()
float c = precise::cos(x); // use precise version of cos()
```

The following symbolic constants are available. Their values are of type `float` and are accurate within the precision of a single precision floating-point number.

Constant name	Description
MAXFLOAT	Value of maximum non-infinite single precision floating-point number.
HUGE_VALF	A positive float constant expression. <code>HUGE_VALF</code> evaluates to +infinity.
INFINITY	A constant expression of type <code>float</code> representing positive or unsigned infinity.
NAN	A constant expression of type <code>float</code> representing a quiet NaN.

The following symbolic constants are also available. Their values are of type `half` and are accurate within the precision of a half precision floating-point number.

Constant name	Description
MAXHALF	Value of maximum non-infinite half precision floating-point number.
HUGE_VALH	A positive half constant expression. HUGE_VALH evaluates to +infinity.

The following constants are also available. They are of type `float` and are accurate within the precision of a single precision floating-point number.

Constant	Description
M_E_F	Value of e
M_LOG2E_F	Value of $\log_2 e$
M_LOG10E_F	Value of $\log_{10} e$
M_LN2_F	Value of $\log_e 2$
M_LN10_F	Value of $\log_e 10$
M_PI_F	Value of π
M_PI_2_F	Value of $\pi / 2$
M_PI_4_F	Value of $\pi / 4$
M_1_PI_F	Value of $1 / \pi$
M_2_PI_F	Value of $2 / \pi$
M_2_SQRTPI_F	Value of $2 / \sqrt{\pi}$
M_SQRT2_F	Value of $\sqrt{2}$
M_SQRT1_2_F	Value of $1 / \sqrt{2}$

The following constants are also available. They are of type `half` and are accurate within the precision of a half precision floating-point number.

Constant	Description
M_E_H	Value of e
M_LOG2E_H	Value of $\log_2 e$
M_LOG10E_H	Value of $\log_{10} e$
M_LN2_H	Value of $\log_e 2$
M_LN10_H	Value of $\log_e 10$
M_PI_H	Value of π
M_PI_2_H	Value of $\pi / 2$

Constant	Description
M_PI_4_H	Value of $\pi / 4$
M_1_PI_H	Value of $1 / \pi$
M_2_PI_H	Value of $2 / \pi$
M_2_SQRTPI_H	Value of $2 / \sqrt{\pi}$
M_SQRT2_H	Value of $\sqrt{2}$
M_SQRT1_2_H	Value of $1 / \sqrt{2}$

5.6 Matrix Functions

The functions in Table 16 are in the Metal standard library and are defined in the header `<metal_matrix>`. T is `float` or `half`.

Table 16 Matrix Functions in the Metal Standard Library

Built-in matrix functions	Description
<code>float determinant(floatn xn)</code> <code>half determinant(halfn xn)</code>	Compute the determinant of the matrix. The matrix must be a square matrix.
<code>floatmxn transpose(floatn xm)</code> <code>halfmxn transpose(halfn xm)</code>	Transpose a matrix.

Example:

```
float4x4 mA;
float det = determinant(mA);
```

5.7 Geometric Functions

The functions in Table 17 are in the Metal standard library and are defined in the header `<metal_geometric>`. T is a vector floating-point type (`floatn` or `halfn`). T_s refers to the corresponding scalar type (i.e. `float` if T is `floatn` and `half` if T is `halfn`).

Table 17 Geometric Functions in the Metal Standard Library

Built-in geometric functions	Description
<code>T cross(T x, T y)</code>	Return the cross product of x and y. T must be a 3-component vector type.

Built-in geometric functions	Description
<code>T_s distance(T x, T y)</code>	Return the distance between x and y, i.e., <code>length(x-y)</code>
<code>T_s distance_squared(T x, T y)</code>	Return the square of the distance between x and y.
<code>T_s dot(T x, T y)</code>	Return the dot product of x and y, i.e., <code>x[0] * y[0] + x[1] * y[1] + ...</code>
<code>T faceforward(T N, T I, T Nref)</code>	If <code>dot(Nref, I) < 0.0</code> return N, otherwise return <code>-N</code> .
<code>T_s length(T x)</code>	Return the length of vector x, i.e., <code>sqrt(x[0]² + x[1]² + ...)</code>
<code>T_s length_squared(T x)</code>	Return the square of the length of vector x, i.e., <code>(x[0]² + x[1]² + ...)</code>
<code>T normalize(T x)</code>	Returns a vector in the same direction as x but with a length of 1.
<code>T reflect(T I, T N)</code>	For the incident vector I and surface orientation N, returns the reflection direction: <code>I - 2 * dot(N, I) * N</code> In order to achieve the desired result, N must be normalized.
<code>T refract(T I, T N, T_s eta)</code>	For the incident vector I and surface normal N, and the ratio of indices of refraction eta, return the refraction vector. The input parameters for the incident vector I and the surface normal N must already be normalized to get the desired results.

For single precision floating-point, Metal also supports a precise and fast variant of the following geometric functions: `distance`, `length` and `normalize`. The `ffast-math` compiler option (refer to section 6.2) is used to select the appropriate variant when compiling the Metal source. In addition, the `metal::precise` and `metal::fast` nested namespaces are also available and provide developers a way to explicitly select the fast or precise variant of these geometric functions.

5.8 Compute Functions

The functions in section 5.8 and its subsections can only be called from a `kernel` function and are defined in the header `<metal_compute>`.

5.8.1 Threadgroup Synchronization Functions

The threadgroup function in Table 18 is supported.

Table 18 Threadgroup Synchronization Compute Function in the Metal Standard Library

Built-in threadgroup function	Description
<code>void threadgroup_barrier(mem_flags flags)</code>	All threads in a threadgroup executing the kernel must execute this function before any thread is allowed to continue execution beyond the <code>threadgroup_barrier</code> .
<code>void simdgroup_barrier(mem_flags flags)</code>	All threads in a SIMDgroup executing the kernel must execute this function before any thread is allowed to continue execution beyond the <code>simdgroup_barrier</code> .

The `threadgroup_barrier` function acts as an execution and memory barrier. The `threadgroup_barrier` function must be encountered by all threads in a threadgroup executing the kernel.

If `threadgroup_barrier` is inside a conditional statement and if any thread enters the conditional statement and executes the barrier, then all threads in the threadgroup must enter the conditional and execute the barrier.

If `threadgroup_barrier` is inside a loop, for each iteration of the loop, all threads in the threadgroup must execute the `threadgroup_barrier` before any threads are allowed to continue execution beyond the `threadgroup_barrier`.

The `threadgroup_barrier` function can also queue a memory fence (reads and writes) to ensure correct ordering of memory operations to threadgroup or device memory.

The `simdgroup_barrier` function acts as an execution and memory barrier. The `simdgroup_barrier` function must be encountered by all threads in a SIMD-group executing the kernel.

If `simdgroup_barrier` is inside a conditional statement and if any thread enters the conditional statement and executes the barrier, then all threads in the SIMDgroup must enter the conditional and execute the barrier.

If `simdgroup_barrier` is inside a loop, for each iteration of the loop, all threads in the SIMDgroup must execute the `simdgroup_barrier` before any threads are allowed to continue execution beyond the `simdgroup_barrier`.

The `simdgroup_barrier` function can also queue a memory fence (reads and writes) to ensure correct ordering of memory operations to threadgroup or device memory

The `mem_flags` argument to `threadgroup_barrier` and `simdgroup_barrier` is a bitfield and can be one or more of the following values, as described in Table 19.

Table 19 `mem_flags` Enum Values for `threadgroup_barrier()`

<code>mem_flags</code>	Description
<code>mem_none</code>	In this case, no memory fence is applied, and <code>threadgroup_barrier</code> acts only as an execution barrier.

mem_flags	Description
mem_device	Ensure correct ordering of memory operations to device memory.
mem_threadgroup	Ensure correct ordering of memory operations to threadgroup memory for threads in a threadgroup.
mem_texture	Ensure correct ordering of memory operations to texture memory for threads in a threadgroup.

The enumeration types used by `mem_flags` are specified as follows:

```
enum class mem_flags { mem_none,
                       mem_device,
                       mem_threadgroup,
                       mem_texture
};
```

5.9 Graphics Functions

This section and its subsections list the set of graphics functions that can be called by a fragment and vertex functions. These are defined in the header `<metal_graphics>`.

5.9.1 Fragment Functions

The functions in this section (listed in Table 20, Table 21, and Table 22) can only be called inside a fragment function (a function declared with the `fragment` qualifier) or inside a function called from a fragment function. Otherwise the behavior is undefined and may result in a compile-time error.

Fragment function helper threads may be created to help evaluate derivatives (explicit or implicit) for use with a fragment thread(s). Fragment function helper threads execute the same code as the non-helper fragment threads, but will not have side effects that modify the render target(s) or any other memory that can be accessed by the fragment function. In particular:

- Fragments corresponding to helper threads are discarded when the fragment function execution is complete without any updates to the render target(s).
- Stores and atomic operations to buffers and textures performed by helper threads have no effect on the underlying memory associated with the buffer or texture.

5.9.1.1 Fragment Functions – Derivatives

Metal includes the functions in Table 20 to compute derivatives. `T` is one of `float`, `float2`, `float3`, `float4`, `half`, `half2`, `half3` or `half4`.

NOTE: Derivatives are undefined within non-uniform control flow.

Table 20 Derivatives Fragment Functions in the Metal Standard Library

Built-in fragment functions	Description
<code>T dfdx(T p)</code>	Returns a high precision partial derivative of the specified value with respect to the screen space x coordinate.
<code>T dfdy(T p)</code>	Returns a high precision partial derivative of the specified value with respect to the screen space y coordinate.
<code>T fwidth(T p)</code>	Returns the sum of the absolute derivatives in x and y using local differencing for p; i.e. <code>fabs(dfdx(p)) + fabs(dfdy(p))</code>

5.9.1.2 Fragment Functions – Samples¹⁷

Metal includes the following per-sample functions in Table 21.

Table 21 Samples Fragment Functions in the Metal Standard Library

Built-in fragment functions	Description
<code>uint get_num_samples()</code>	Returns the number of samples for the multisampled color attachment.
<code>float2 get_sample_position(uint indx)</code>	Returns the normalized sample offset (x, y) for a given sample index <code>indx</code> . Values of x and y are in [-1.0 ... 1.0].

5.9.1.3 Fragment Functions – Flow Control

The Metal function in Table 22 is used to terminate a fragment.

Table 22 Fragment Flow Control Function in the Metal Standard Library

Built-in fragment functions	Description
<code>void discard_fragment(void)</code>	Marks the current fragment as being terminated, and the output of the fragment function for this fragment is discarded.

NOTE:

¹⁷ `get_num_samples` and `get_sample_position` return the number of samples for the color attachment and the sample offsets for a given sample index. For example, this can be used to shade per-fragment but do the alpha test per-sample for transparency super-sampling.

- Writes to a buffer or texture from a fragment thread before **discard_fragment** is called will not be discarded.
- Multiple fragment threads or helper threads associated with a fragment thread execute together to compute derivatives. If any (but not all) of these threads executes the **discard_fragment** function, the behavior of any following derivative computations (explicit or implicit) is undefined.

5.10 Texture Functions

The texture member functions are categorized into: sample from a texture, sample compare from a texture, read (sampler-less read) from a texture, gather from a texture, gather compare from a texture, write to a texture, texture query and texture fence functions.

These are defined in the header `<metal_texture>`.

The texture **sample**, **sample_compare**, **gather** and **gather_compare** functions take an **offset** argument for a 2D texture, 2D texture array and 3D texture. The **offset** is an integer value that is applied to the texture coordinate before looking up each pixel. This integer value can be in the range -8 to +7. The default value is 0.

Overloaded variants of texture **sample** and **sample_compare** functions for a 2D texture, 2D texture array, 3D texture, and cube are available and allow the texture to be sampled using a bias that is applied to a mip-level before sampling or with user-provided gradients in the x and y direction.

NOTE:

- The texture **sample**, **sample_compare**, **gather** and **gather_compare** functions require that the texture is declared with the **sample** access qualifier.
- The texture **sample_compare** and **gather_compare** functions are only available for depth texture types.
- The texture **read** functions require that the texture is declared with the **sample**, **read** or **read_write** access qualifier.
- The texture **write** functions require that the texture is declared with the **write** or **read_write** access qualifier.
- The **sample** and **sample_compare** functions that do not take an explicit LOD or gradients and the **gather** and **gather_compare** functions when called from kernel or vertex functions assume an implicit LOD of 0.
- For the **gather** and **gather_compare** functions, the four samples that would contribute to filtering are placed into xyzw in counter clockwise order starting with the sample to the lower left of the queried location. This is the same as nearest sampling with un-normalized texture coordinate deltas at the following locations: $(-,+)$, $(+,+)$, $(+,-)$, $(-,-)$, where the magnitude of the deltas are always half a texel.

5.10.1 1D Texture

The following member functions can be used to sample from a 1D texture.

```
T_v18    sample(sampler s, float coord) const
```

The following member functions can be used to perform sampler-less reads from a 1D texture:

```
T_v      read(uint coord, uint lod = 0) const
T_v      read(ushort coord, ushort lod = 0) const
```

The following member functions can be used to write to a specific mip-level of a 1D texture.

```
void     write(T_v color, uint coord, uint lod = 0)
void     write(T_v color, ushort coord, ushort lod = 0)
```

The following 1D texture query member functions are provided.

```
uint     get_width(uint lod = 0) const
uint     get_num_mip_levels() const
```

5.10.2 1D Texture Array

The following member functions can be used to sample from a 1D texture array.

```
T_v      sample(sampler s, float coord, uint array) const
```

The following member functions can be used to perform sampler-less reads from a 1D texture array:

```
T_v      read(uint coord, uint array, uint lod = 0) const
T_v      read(ushort coord, ushort array, ushort lod = 0) const
```

The following member functions can be used to write to a specific mip-level of a 1D texture array.

```
void     write(T_v color, uint coord, uint array, uint lod = 0)
void     write(T_v color, ushort coord, ushort array, ushort lod = 0)
```

The following 1D texture array query member functions are provided.

```
uint     get_width(uint lod = 0) const
uint     get_array_size() const
uint     get_num_mip_levels() const
```

5.10.3 2D Texture

¹⁸ T_v is a 4-component vector type based on the templated type <T> used to declare the texture type. If T is float, T_v is float4. If T is half, T_v is half4. If T is int, T_v is int4. If T is uint, T_v is uint4. If T is short, T_v is short4 and if T is ushort, T_v is ushort4.

The following data types and corresponding constructor functions are available to specify various sampling options:

```
bias(float value)
level(float lod)
gradient2d(float2 dPdx, float2 dPdy)
```

The following member functions can be used to sample from a 2D texture.

```
Tv    sample(sampler s, float2 coord, int2 offset = int2(0)) const
Tv    sample(sampler s, float2 coord, lod_options options,
           int2 offset = int2(0)) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradient2d`.

The following member functions can be used to perform sampler-less reads from a 2D texture:

```
Tv    read(uint2 coord, uint lod = 0) const
Tv    read(ushort2 coord, ushort lod = 0) const
```

The following member functions can be used to write to a 2D texture.

```
void  write(Tv color, uint2 coord, uint lod = 0)
void  write(Tv color, ushort2 coord, ushort lod = 0)
```

The following member functions can be used to do a gather of four samples that would be used for bilinear interpolation when sampling a 2D texture.

```
enum class component { x, y, z, w };
Tv    gather(sampler s, float2 coord, int2 offset = int2(0),
           component c = component::x) const
```

The following 2D texture query member functions are provided.

```
uint  get_width(uint lod = 0) const
uint  get_height(uint lod = 0) const
uint  get_num_mip_levels()const
```

5.10.3.1 2D Texture Sampling Example

The following code shows several uses of the 2D texture sample function, depending upon its arguments.

```
#include <metal_stdlib>
using namespace metal;
```

```

texture2d<float> tex;
sampler s;
float2 coord;
int2 offset;
float lod;

// no optional arguments
float4 clr = tex.sample(s, coord);

// sample using a mip-level
clr = tex.sample(s, coord, level(lod));

// sample with an offset
clr = tex.sample(s, coord, offset);

// sample using a mip-level and an offset
clr = tex.sample(s, coord, level(lod), offset);

```

5.10.4 2D Texture Array

The following member functions can be used to sample from a 2D texture array.

```

T_v    sample(sampler s, float2 coord, uint array,
              int2 offset = int2(0)) const
T_v    sample(sampler s, float2 coord, uint array, lod_options options,
              int2 offset = int2(0)) const

```

`lod_options` must be one of the following types: `bias`, `level` or `gradient2d`.

The following member functions can be used to perform sampler-less reads from a 2D texture array:

```

T_v    read(uint2 coord, uint array, uint lod = 0) const
T_v    read(ushort2 coord, ushort array, ushort lod = 0) const

```

The following member functions can be used to write to a 2D texture array.

```

void    write(T_v color, uint2 coord, uint array, uint lod = 0)
void    write(T_v color, ushort2 coord, ushort array, ushort lod = 0)

```

The following member functions can be used to do a gather of four samples that would be used for bilinear interpolation when sampling a 2D texture array.

```

T_v    gather(sampler s, float2 coord, uint array,

```

```
int2 offset = int2(0), component c = component::x) const
```

The following 2D texture array query member functions are provided.

```
uint get_width(uint lod = 0) const
uint get_height(uint lod = 0) const
uint get_array_size() const
uint get_num_mip_levels() const
```

5.10.5 3D Texture

The following data types and corresponding constructor functions are available to specify various sampling options:

```
bias(float value)
level(float lod)
gradient3d(float3 dPdx, float3 dPdy)
```

The following member functions can be used to sample from a 3D texture.

```
T_v sample(sampler s, float3 coord, int3 offset = int3(0)) const
T_v sample(sampler s, float3 coord, lod_options options,
           int3 offset = int3(0)) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradient3d`.

The following member functions can be used to perform sampler-less reads from a 3D texture:

```
T_v read(uint3 coord, uint lod = 0) const
T_v read(ushort3 coord, ushort lod = 0) const
```

The following member functions can be used to write to a 3D texture.

```
void write(T_v color, uint3 coord, uint lod = 0)
void write(T_v color, ushort3 coord, ushort lod = 0)
```

The following 3D texture query member functions are provided.

```
uint get_width(uint lod = 0) const
uint get_height(uint lod = 0) const
uint get_depth(uint lod = 0) const
uint get_num_mip_levels() const
```

5.10.6 Cube Texture

The following data types and corresponding constructor functions are available to specify various sampling options:

```
bias(float value)
level(float lod)
gradientcube(float3 dPdx, float3 dPdy)
```

The following member functions can be used to sample from a cube texture.

```
T_v    sample(sampler s, float3 coord) const
T_v    sample(sampler s, float3 coord, lod_options options) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradientcube`.

NOTE: Table 23 describes the cube face and the number used to identify the face.

Table 23 Cube Face Number

Face number	Cube face
0	Positive X
1	Negative X
2	Positive Y
3	Negative Y
4	Positive Z
5	Negative Z

The following member functions can be used to do a gather of four samples that would be used for bilinear interpolation when sampling a cube texture.

```
T_v    gather(sampler s, float3 coord, component c = component::x) const
```

The following member functions can be used to perform sampler-less reads from a cube texture:

```
T_v    read(uint2 coord, uint face, uint lod = 0) const
T_v    read(ushort2 coord, ushort face, ushort lod = 0) const
```

The following member functions can be used to write to a cube texture.

```
void    write(T_v color, uint2 coord, uint face, uint lod = 0)
void    write(T_v color, ushort2 coord, ushort face, ushort lod = 0)
```

The following cube texture query member functions are provided.

```
uint get_width(uint lod = 0) const
uint get_height(uint lod = 0) const
uint get_num_mip_levels() const
```

5.10.7 Cube Array Texture

The following data types and corresponding constructor functions are available to specify various sampling options:

```
bias(float value)
level(float lod)
gradientcube(float3 dPdx, float3 dPdy)
```

The following member functions can be used to sample from a cube array texture.

```
Tv sample(sampler s, float3 coord, uint array) const
Tv sample(sampler s, float3 coord, uint array,
          lod_options options) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradientcube`.

The following member functions can be used to do a gather of four samples that would be used for bilinear interpolation when sampling a cube array texture.

```
Tv gather(sampler s, float3 coord, uint array,
          component c = component::x) const
```

The following member functions can be used to perform sampler-less reads from a cube array texture:

```
Tv read(uint2 coord, uint face, uint array, uint lod = 0) const
Tv read(ushort2 coord, ushort face, ushort array,
        ushort lod = 0) const
```

The following member functions can be used to write to a cube array texture.

```
void write(Tv color, uint2 coord, uint face, uint array, uint lod = 0)
void write(Tv color, ushort2 coord, ushort face, ushort array,
          ushort lod = 0)
```

The following cube array texture query member functions are provided.

```
uint get_width(uint lod = 0) const
uint get_height(uint lod = 0) const
```



```
uint get_array_size() const
uint get_num_mip_levels() const
```

5.10.8 2D Multisampled Texture

The following member functions can be used to perform sampler-less reads from a 2D multisampled texture:

```
Tv    read(uint2 coord, uint sample) const
T      read(ushort2 coord, ushort sample) const
```

The following 2D multisampled texture query member functions are provided.

```
uint get_width() const
uint get_height() const
uint get_num_samples() const
```

5.10.9 2D Depth Texture

The following data types and corresponding constructor functions are available to specify various sampling options:

```
bias(float value)
level(float lod)
gradient2d(float2 dPdx, float2 dPdy)
```

The following member functions can be used to sample from a 2D depth texture.

```
T      sample(sampler s, float2 coord, int2 offset = int2(0)) const
T      sample(sampler s, float2 coord, lod_options options,
              int2 offset = int2(0)) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradient2d`.

The following member functions can be used to sample from a 2D depth texture and compare a single component against the specified comparison value

```
T      sample_compare(sampler s, float2 coord, float compare_value,
                      int2 offset = int2(0)) const

T      sample_compare(sampler s, float2 coord, float compare_value,
                      lod_options options, int2 offset = int2(0)) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradient2d`. `T` must be a `float` type.

NOTE: `sample_compare` performs a comparison of the `compare_value` value against the pixel value (1.0 if the comparison passes and 0.0 if it fails). These comparison result values per-pixel are

then blended together as in normal texture filtering and the resulting value between 0.0 and 1.0 is returned.

The following member functions can be used to perform sampler-less reads from a 2D depth texture:

```
T    read(uint2 coord, uint lod = 0) const
T    read(ushort2 coord, ushort lod = 0) const
```

The following built-in functions can be used to do a gather of four samples that would be used for bilinear interpolation when sampling a 2D depth texture.

```
T_v    gather(sampler s, float2 coord, int2 offset = int2(0)) const
```

The following member functions can be used to do a gather of four samples that would be used for bilinear interpolation when sampling a 2D depth texture and comparing these samples with a specified comparison value (1.0 if the comparison passes and 0.0 if it fails).

```
T_v    gather_compare(sampler s, float2 coord, float compare_value,
                      int2 offset = int2(0)) const
```

T must be a float type.

The following 2D depth texture query member functions are provided.

```
uint    get_width(uint lod = 0) const
uint    get_height(uint lod = 0) const
uint    get_num_mip_levels() const
```

5.10.10 2D Depth Texture Array

The following member functions can be used to sample from a 2D depth texture array.

```
T    sample(sampler s, float2 coord, uint array,
            int2 offset = int2(0)) const
T    sample(sampler s, float2 coord, uint array, lod_options options,
            int2 offset = int2(0)) const
```

lod_options must be one of the following types: bias, level or gradient2d.

The following member functions can be used to sample from a 2D depth texture array and compare a single component against the specified comparison value

```
T    sample_compare(sampler s, float2 coord, uint array,
                    float compare_value, int2 offset = int2(0)) const
T    sample_compare(sampler s, float2 coord, uint array,
                    float compare_value, lod_options options,
```

```
int2 offset = int2(0)) const
```

lod_options must be one of the following types: bias, level or gradient2d. T must be a float type.

The following member functions can be used to perform sampler-less reads from a 2D depth texture array:

```
T      read(uint2 coord, uint array, uint lod = 0) const
T      read(ushort2 coord, ushort array, ushort lod = 0) const
```

The following member functions can be used to do a gather of four samples that would be used for bilinear interpolation when sampling a 2D depth texture array.

```
T_v    gather(sampler s, float2 coord, uint array,
              int2 offset = int2(0)) const
```

The following member functions can be used do a gather of four samples that would be used for bilinear interpolation when sampling a 2D depth texture array and comparing these samples with a specified comparison value.

```
T_v    gather_compare(sampler s, float2 coord, uint array,
                     float compare_value, int2 offset = int2(0)) const
```

T must be a float type.

The following 2D depth texture array query member functions are provided.

```
uint    get_width(uint lod = 0) const
uint    get_height(uint lod = 0) const
uint    get_array_size() const
uint    get_num_mip_levels() const
```

5.10.11 Cube Depth Texture

The following data types and corresponding constructor functions are available to specify various sampling options:

```
bias(float value)
level(float lod)
gradientcube(float3 dPdx, float3 dPdy)
```

The following member functions can be used to sample from a cube depth texture.

```
T      sample(sampler s, float3 coord) const
T      sample(sampler s, float3 coord, lod_options options) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradientcube`.

The following member functions can be used to sample from a cube depth texture and compare a single component against the specified comparison value

```
T    sample_compare(sampler s, float3 coord,
                    float compare_value) const
T    sample_compare(sampler s, float3 coord,
                    float compare_value, lod_options options) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradientcube`. `T` must be a `float` type.

The following member functions can be used to perform sampler-less reads from a cube depth texture:

```
T    read(uint2 coord, uint face, uint lod = 0) const
T    read(ushort2 coord, ushort face, ushort lod = 0) const
```

The following member functions can be used to do a gather of four samples that would be used for bilinear interpolation when sampling a cube depth texture.

```
T_v    gather(sampler s, float3 coord) const
```

The following member functions can be used do a gather of four samples that would be used for bilinear interpolation when sampling a cube texture and comparing these samples with a specified comparison value.

```
T_v    gather_compare(sampler s, float3 coord,
                      float compare_value) const
```

`T` must be a `float` type.

The following cube depth texture query member functions are provided.

```
uint    get_width(uint lod = 0) const
uint    get_height(uint lod = 0) const
uint    get_num_mip_levels() const
```

5.10.12 Cube Array Depth Texture

The following data types and corresponding constructor functions are available to specify various sampling options:

```
bias(float value)
level(float lod)
gradientcube(float3 dPdx, float3 dPdy)
```

The following member functions can be used to sample from a cube array depth texture.

```
T    sample(sampler s, float3 coord, uint array) const
T    sample(sampler s, float3 coord, uint array,
           lod_options options) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradientcube`.

The following member functions can be used to sample from a cube depth texture and compare a single component against the specified comparison value

```
T    sample_compare(sampler s, float3 coord, uint array,
                   float compare_value) const
T    sample_compare(sampler s, float3 coord, uint array,
                   float compare_value, lod_options options) const
```

`lod_options` must be one of the following types: `bias`, `level` or `gradientcube`. `T` must be a `float` type.

The following member functions can be used to perform sampler-less reads from a cube depth texture array:

```
T    read(uint2 coord, uint face, uint array, uint lod = 0) const
T    read(ushort2 coord, ushort face, ushort array,
           ushort lod = 0) const
```

The following member functions can be used to do a gather of four samples that would be used for bilinear interpolation when sampling a cube depth texture.

```
T_v    gather(sampler s, float3 coord, uint array) const
```

The following member functions can be used do a gather of four samples that would be used for bilinear interpolation when sampling a cube texture and comparing these samples with a specified comparison value.

```
T_v    gather_compare(sampler s, float3 coord, uint array,
                     float compare_value) const
```

`T` must be a `float` type.

The following cube depth texture query member functions are provided.

```
uint    get_width(uint lod = 0) const
uint    get_height(uint lod = 0) const
uint    get_array_size() const
uint    get_num_mip_levels() const
```

5.10.13 2D Multisampled Depth Texture

The following member functions can be used to perform sampler-less reads from a 2D multisampled depth texture:

```
T    read(uint2 coord, uint sample) const
T    read(ushort2 coord, ushort sample) const
```

The following 2D multisampled depth texture query member functions are provided.

```
uint  get_width() const
uint  get_height() const
uint  get_num_samples() const
```

5.10.14 Texture Fence Functions

The following member function is supported by texture types that can be declared with the `access::read_write` qualifier:

```
void  fence()
```

The texture fence function ensures that writes to the texture by a thread become visible to subsequent reads from that texture by the same thread (i.e. the thread performing the write).

The following example show how the texture fence function can be used to make sure that writes to a texture by a thread are visible to later reads to the same location by the same thread.

```
kernel void
foo(texture2d<float, access::read_write> texA,
    "",
    ushort2 gid [[ thread_position_in_grid ]])
{
    float4 clr = ...;
    texA.write(gid, clr);

    ""
    // fence to ensure that writes by thread become
    // visible to later reads by thread
    texA.fence();

    clr_new = texA.read(gid);
    ""
}
```

5.10.15 Null Texture Functions

The following function can be used to determine if a texture is a null texture.

```
bool  is_null_texture(texture1d<T, access>);
bool  is_null_texture(texture1d_array<T, access>);
bool  is_null_texture(texture2d<T, access>);
bool  is_null_texture(texture2d_array<T, access>);
```

```

bool  is_null_texture(texture3d<T, access>);
bool  is_null_texture(texturecube<T, access>);
bool  is_null_texture(texturecube_array<T, access>);
bool  is_null_texture(texture2d_ms<T, access>);
bool  is_null_texture(depth2d<T, access>);
bool  is_null_texture(depth2d_array<T, access>);
bool  is_null_texture(depthcube<T, access>);
bool  is_null_texture(depthcube_array<T, access>);
bool  is_null_texture(depth2d_ms<T, access>);

```

Returns `true` if the texture is a null texture and `false` otherwise. The behavior of calling any texture member function with a null texture is undefined.

5.11 Pack and Unpack Functions

This section lists the Metal functions for converting a vector floating-point data to and from a packed integer value. The functions are defined in the header `<metal_pack>`. Refer to section 7.7 for details on how to convert from a 8-bit, 10-bit or 16-bit signed or unsigned integer value to a normalized single- or half-precision floating-point value and vice-versa.

5.11.1 Unpack Integer(s); Convert to a Floating-Point Vector

Table 24 lists functions that unpack multiple values from a single unsigned integer and then converts them into floating-point values that are stored in a vector.

Table 24 Unpack Functions

Built-in unpack functions	Description
float4 unpack_unorm4x8_to_float(uint x) float4 unpack_snorm4x8_to_float(uint x) half4 unpack_unorm4x8_to_half(uint x) half4 unpack_snorm4x8_to_half(uint x)	Unpack a 32-bit unsigned integer into four 8-bit signed or unsigned integers and then convert each 8-bit signed or unsigned integer value to a normalized single- or half-precision floating-point value to generate a 4-component vector.
float4 unpack_unorm4x8_srgb_to_float(uint x) half4 unpack_unorm4x8_srgb_to_half(uint x)	Unpack a 32-bit unsigned integer into four 8-bit signed or unsigned integers and then convert each 8-bit signed or unsigned integer value to a normalized single- or half-precision floating-point value to generate a 4-component vector. The r, g, and b color values are converted from sRGB to linear RGB.

Built-in unpack functions	Description
float2 unpack_unorm2x16_to_float(uint x) float2 unpack_snorm2x16_to_float(uint x) half2 unpack_unorm2x16_to_half(uint x) half2 unpack_snorm2x16_to_half(uint x)	Unpack a 32-bit unsigned integer into two 16-bit signed or unsigned integers and then convert each 16-bit signed or unsigned integer value to a normalized single- or half-precision floating-point value to generate a 2-component vector.
float4 unpack_unorm10a2_to_float(uint x) float3 unpack_unorm565_to_float(ushort x) half4 unpack_unorm10a2_to_half(uint x) half3 unpack_unorm565_to_half(ushort x)	Convert a 1010102 (10a2), or 565 color value to the corresponding normalized single- or half-precision floating-point vector.

5.11.2 Convert Floating-Point Vector to Integers, then Pack the Integers

Table 25 lists functions that start with a floating-point vector, converts the components into integer values, and then packs the multiple values into a single unsigned integer.

Table 25 Pack Functions

Built-in pack functions	Description
uint pack_float_to_unorm4x8(float4 x) uint pack_float_to_snorm4x8(float4 x) uint pack_half_to_unorm4x8(half4 x) uint pack_half_to_snorm4x8(half4 x)	Convert a 4-component vector normalized single- or half-precision floating-point value to four 8-bit integer values and pack these 8-bit integer values into a 32-bit unsigned integer.
uint pack_float_to_srgb_unorm4x8(float4 x) uint pack_half_to_srgb_unorm4x8(half4 x)	Convert a 4-component vector normalized single- or half-precision floating-point value to four 8-bit integer values and pack these 8-bit integer values into a 32-bit unsigned integer. The color values are converted from linear RGB to sRGB.
uint pack_float_to_unorm2x16(float2 x) uint pack_float_to_snorm2x16(float2 x) uint pack_half_to_unorm2x16(half2 x) uint pack_half_to_snorm2x16(half2 x)	Convert a 2-component vector of normalized single- or half-precision floating-point values to two 16-bit integer values and pack these 16-bit integer values into a 32-bit unsigned integer.
uint pack_float_to_unorm10a2(float4) ushort pack_float_to_unorm565(float3) uint pack_half_to_unorm10a2(half4) ushort pack_half_to_unorm565(half3)	Convert a 4- or 3-component vector of normalized single- or half-precision floating-point values to a packed, 1010102 or 565 color integer value.

5.12 Atomic Functions

The Metal programming language implements a subset of the C++14 atomics and synchronization operations. For atomic operations, only a `memory_order` of `memory_order_relaxed` is supported. If the atomic operation is to threadgroup memory, the memory scope of these atomic operations is a threadgroup. If the atomic operation is to device memory, then the memory scope is the GPU device.

There are only a few kinds of operations on atomic types, although there are many instances of those kinds. This section specifies each general kind.

These are defined in the header `<metal_atomic>`.

NOTE: Atomic operations to device and threadgroup memory can only be performed inside a kernel function (a function declared with the **kernel** qualifier) or inside a function called from a kernel function.

The `memory_order` enum is defined as:

```
enum memory_order {  
    memory_order_relaxed19  
};
```

5.12.1 Atomic Store Functions

These functions atomically replace the value pointed to by `object` with `desired`.

```
void  atomic_store_explicit(volatile device atomic_int* object,  
                           int desired, memory_order order)  
void  atomic_store_explicit(volatile device atomic_uint* object,  
                           uint desired, memory_order order)  
void  atomic_store_explicit(volatile threadgroup atomic_int* object,  
                           int desired, memory_order order)  
void  atomic_store_explicit(volatile threadgroup atomic_uint* object,  
                           uint desired, memory_order order)
```

5.12.2 Atomic Load Functions

These functions atomically obtain the value pointed to by `object`.

```
int    atomic_load_explicit(volatile device atomic_int* object,  
                           memory_order order)  
uint   atomic_load_explicit(volatile device atomic_uint* object,  
                           memory_order order)  
int    atomic_load_explicit(volatile threadgroup atomic_int* object,  
                           memory_order order)
```

¹⁹ Only `memory_order_relaxed` is supported in this revision of Metal.

```
uint atomic_load_explicit(volatile threadgroup atomic_uint* object,
                          memory_order order)
```

5.12.3 Atomic Exchange Functions

These functions atomically replace the value pointed to by `object` with `desired` and return the value `object` previously held.

```
int atomic_exchange_explicit(volatile device atomic_int *object,
                             int desired, memory_order order)
uint atomic_exchange_explicit(volatile device atomic_uint *object,
                              uint desired, memory_order order)
int atomic_exchange_explicit(volatile threadgroup atomic_int *object,
                             int desired, memory_order order)
uint atomic_exchange_explicit(volatile threadgroup atomic_uint
                              *object, uint desired,
                              memory_order order)
```

5.12.4 Atomic Compare and Exchange functions

These functions atomically compare the value pointed to by `object` with the value in `expected`. If those values are equal, the function replaces `*object` with `desired` (by performing a read-modify-write operation). The function returns the value `object` previously held.

```
bool atomic_compare_exchange_weak_explicit(
    volatile device atomic_int *object,
    int *expected, int desired,
    memory_order succ, memory_order fail)
bool atomic_compare_exchange_weak_explicit(
    volatile device atomic_uint *object,
    uint *expected, uint desired,
    memory_order succ, memory_order fail)
bool atomic_compare_exchange_weak_explicit(
    volatile threadgroup atomic_int *object,
    int *expected, int desired,
    memory_order succ, memory_order fail)
bool atomic_compare_exchange_weak_explicit(
    volatile threadgroup atomic_uint *object,
    uint *expected, uint desired,
    memory_order succ, memory_order fail)
```

NOTE: The effect of the compare-and-exchange operations is:

```

if (memcmp(object, expected, sizeof(*object) == 0)
    memcpy(object, &desired, sizeof(*object));
else
    memcpy(expected, object, sizeof(*object));

```

5.12.5 Atomic Fetch and Modify functions

The following operations perform arithmetic and bitwise computations. All of these operations are applicable to an object of any atomic type. The key, operator, and computation correspondence is given in Table 26.

Table 26 Atomic Operation Function

key	operator	computation
add	+	addition
and	&	bitwise and
max	max	compute max
min	min	compute min
or		bitwise inclusive or
sub	–	subtraction
xor	^	bitwise exclusive or

Atomically replaces the value pointed to by `object` with the result of the computation of the value specified by `key` and `operator`. These operations are atomic read-modify-write operations. For signed integer types, arithmetic is defined to use two's complement representation with silent wrap-around on overflow; there are no undefined results. There are no undefined results. Returns the value `object` held previously.

```

int    atomic_fetch_key_explicit(
        volatile device atomic_int *object,
        int arg, memory_order order)
uint   atomic_fetch_key_explicit(
        volatile device atomic_uint *object,
        uint arg, memory_order order)
int     atomic_fetch_key_explicit(
        volatile threadgroup atomic_int *object,
        int arg, memory_order order)
uint    atomic_fetch_key_explicit(
        volatile threadgroup atomic_uint *object,
        uint arg, memory_order order)

```

6 Compiler Options

The Metal compiler can be used online (i.e. using the appropriate APIs to compile Metal sources) or offline. Metal sources compiled offline can be loaded as binaries, using the appropriate Metal APIs.

This chapter explains the compiler options supported by the Metal compiler, which are categorized as pre-processor options, options for math intrinsics, options that control optimization and miscellaneous options. The online and offline Metal compiler support these options.

6.1 Pre-Processor Options

These options control the Metal preprocessor that is run on each program source before actual compilation.

`-D name`

Predefine *name* as a macro, with definition 1.

`-D name=definition`

The contents of *definition* are tokenized and processed as if they appeared in a `#define` directive. This option may receive multiple options, which are processed in the order in which they appear. This option allows developers to compile Metal code to change which features are enabled or disabled.

`-I dir`

Add the directory *dir* to the list of directories to be searched for header files. This option is only available for the offline compiler.

6.2 Math Intrinsics Options

These options control compiler behavior regarding floating-point arithmetic. These options trade off between speed and correctness.

`-ffast-math` (default)
`-fno-fast-math`

These options enable (default) or disable the optimizations for floating-point arithmetic that may violate the IEEE 754 standard. They also enable or disable the high precision variant of math functions for single precision floating-point scalar and vector types.

The optimizations for floating-point arithmetic include:

- No NaNs – Allow optimizations to assume the arguments and result are not NaN.
- No Infs – Allow optimizations to assume the arguments and result are not positive or negative infinity.
- No Signed Zeroes – Allow optimizations to treat the sign of zero argument or result as insignificant.
- Allow Reciprocal – Allow optimizations to use the reciprocal of an argument rather than perform division

- Fast – Allow algebraically equivalent transformations i.e. re-associate floating-point operations that may dramatically change results in floating-point.

6.3 Options Controlling the Language Version

The following option controls the version of unified graphics/compute language that the compiler accepts.

`-std=`

Determine the language revision to use. A value for this option must be provided. This can only be:

- `ios-metal1.0` – support the unified graphics / compute language revision 1.0 programs for iOS 8.0.
- `ios-metal1.1` – support the unified graphics / compute language revision 1.1 programs for iOS 9.0.
- `ios-metal1.2` – support the unified graphics / compute language revision 1.2 programs for iOS 10.0.
- `osx-metal1.1` – support the unified graphics / compute language revision 1.1 programs for OS X 10.11.
- `osx-metal1.2` – support the unified graphics / compute language revision 1.2 programs for macOS 10.12.

6.4 Options to Request or Suppress Warnings

The following options are available.

`-Werror`

Make all warnings into errors.

`-W`

Inhibit all warning messages.

7 Numerical Compliance

This chapter covers how Metal represents floating-point numbers with regard to accuracy in mathematical operations. Metal is compliant to a subset of the IEEE 754 standard.

7.1 INF, NaN and Denormalized Numbers

INF must be supported for single precision floating-point numbers and are optional for half precision floating-point numbers.

NaNs must be supported for single precision floating-point numbers (with fast math disabled) and are optional for half precision floating-point numbers. If fast math is enabled the behavior of handling NaN or INF (as inputs or outputs) is undefined. Signaling NaNs are not supported.

Denormalized single or half precision floating-point numbers passed as input to or produced as the output of single or half precision floating-point arithmetic operations may be flushed to zero.

7.2 Rounding Mode

Either round to nearest even or round to zero rounding mode may be supported for single precision and half precision floating-point operations.

7.3 Floating-point Exceptions

Floating-point exceptions are disabled in Metal.

7.4 Relative Error as ULPs

Table 27 describes the minimum accuracy of single-precision floating-point basic arithmetic operations and math functions given as ULP values. The reference value used to compute the ULP value of an arithmetic operation is the infinitely precise result.

Table 27 Minimum Accuracy of Single Precision Floating-Point Operations and Functions

Math Function	Min Accuracy - ULP values
$x + y$	Correctly rounded
$x - y$	Correctly rounded
$x * y$	Correctly rounded
$1.0 / x$	≤ 1 ulp
x / y	≤ 2.5 ulp
acos	≤ 4 ulp

Math Function	Min Accuracy - ULP values
acosh	≤ 4 ulp
asin	≤ 4 ulp
asinh	≤ 4 ulp
atan	≤ 5 ulp
atan2	≤ 6 ulp
atanh	≤ 5 ulp
ceil	Correctly rounded
copysign	0 ulp
cos	≤ 4 ulp
cosh	≤ 4 ulp
cospi	≤ 4 ulp
exp	≤ 4 ulp
exp2	≤ 4 ulp
exp10	≤ 4 ulp
fabs	0 ulp
fdim	Correctly rounded
floor	Correctly rounded
fma	Correctly rounded
fmax	0 ulp
fmin	0 ulp
fmod	0 ulp
fract	Correctly rounded
frexp	0 ulp
ilogb	0 ulp
ldexp	Correctly rounded
log	≤ 4 ulp
log2	≤ 4 ulp
log10	≤ 4 ulp
modf	0 ulp

Math Function	Min Accuracy - ULP values
pow	≤ 16 ulp
powr	≤ 16 ulp
rint	Correctly rounded
round	Correctly rounded
rsqrt	≤ 2 ulp
sin	≤ 4 ulp
sincos	≤ 4 ulp
sinh	≤ 4 ulp
sinpi	≤ 4 ulp
sqrt	≤ 3 ulp
tan	≤ 6 ulp
tanpi	≤ 6 ulp
tanh	≤ 5 ulp
trunc	Correctly rounded

Table 28 describes the minimum accuracy of single-precision floating-point arithmetic operations given as ULP values with fast math enabled (which is the default unless `-ffast-math-disable` is specified as a compiler option).

Table 28 Minimum Accuracy of Single Precision Operations and Functions with Fast Math Enabled

Math Function	Min Accuracy - ULP values
$x + y$	Correctly rounded
$x - y$	Correctly rounded
$x * y$	Correctly rounded
$1.0 / x$	≤ 1 ulp for x in the domain of 2^{-126} to 2^{126}
x / y	≤ 2.5 ulp for y in the domain of 2^{-126} to 2^{126}
$\text{acos}(x)$	≤ 5 ulp for x in the domain $[-1, 1]$
$\text{acosh}(x)$	Implemented as $\log(x + \sqrt{x * x - 1.0})$
$\text{asin}(x)$	≤ 5 ulp for x in the domain $[-1, 1]$ and $ x \geq 2^{-125}$

asinh(x)	Implemented as $\log(x + \sqrt{x * x + 1.0})$
atan(x)	≤ 5 ulp
atanh(x)	Implemented as $0.5 * (\log(1.0 + x) / \log(1.0 - x))$
atan2(y, x)	Implemented as $\text{atan}(y / x)$ for $x > 0$, $\text{atan}(y / x) + M_PI_F$ for $x < 0$ and $y > 0$, $\text{atan}(y / x) - M_PI_F$ for $x < 0$ and $y < 0$ and is undefined if $y = 0$ and $x = 0$.
cos(x)	For x in the domain $[-\pi, \pi]$, the maximum absolute error is $\leq 2^{-13}$ and larger otherwise.
cosh(x)	Implemented as $0.5 * (\exp(x) + \exp(-x))$
cospi(x)	The maximum relative error is $\leq 2^{-17}$.
exp(x)	$\leq 3 + \text{floor}(\text{fabs}(2 * x))$ ulp
exp2(x)	$\leq 3 + \text{floor}(\text{fabs}(2 * x))$ ulp
exp10(x)	Implemented as $\exp2(x * \log2(10))$
fabs	0 ulp
fdim	Correctly rounded
floor	Correctly rounded
fma	Correctly rounded
fmax	0 ulp
fmin	0 ulp
fmod	0 ulp
fract	Correctly rounded
frexp	0 ulp
ilogb	0 ulp
ldexp	Correctly rounded
log(x)	For x in the domain $[0.5, 2]$, the maximum absolute error is $\leq 2^{-21}$; otherwise the maximum error is ≤ 3 ulp if $x > 0$; otherwise the results are undefined.
log2(x)	For x in the domain $[0.5, 2]$, the maximum absolute error is $\leq 2^{-22}$; otherwise the maximum error is ≤ 2 ulp if $x > 0$; otherwise the results are undefined.
log10(x)	Implemented as $\log2(x) * \log10(2)$
modf	0 ulp
pow(x, y)	Implemented as $\exp2(y * \log2(x))$. Undefined for $x = 0$ and $y = 0$.
powr(x, y)	Implemented as $\exp2(y * \log2(x))$. Undefined for $x = 0$ and $y = 0$.
rint	Correctly rounded

round(x)	Correctly rounded
rsqrt	≤ 2 ulp
sin(x)	For x in the domain $[-\pi, \pi]$, the maximum absolute error is $\leq 2^{-13}$ and larger otherwise.
sinh(x)	Implemented as $0.5 * (\exp(x) - \exp(-x))$
sincos(x)	ULP values as defined for sin(x) and cos(x)
sinpi(x)	The maximum relative error is $\leq 2^{-17}$.
sqrt(x)	Implemented as $x * \text{rsqrt}(x)$ with special cases handled correctly.
tan(x)	Implemented as $\sin(x) * (1.0 / \cos(x))$
tanh(x)	Implemented as $(t - 1.0)/(t + 1.0)$ where $t = \exp(2.0 * x)$
tanpi(x)	The maximum relative error is $\leq 2^{-17}$.
trunc	Correctly rounded

NOTE: Even though the precision of individual math operations and functions are specified in table 28 above, the Metal compiler, in fast math mode, may re-associate floating-point operations that may dramatically change results in floating-point. Re-association may change or ignore the sign of zero, allow optimizations to assume the arguments and result are not NaN or +/-INF, inhibit or create underflow or overflow and thus cannot be used by code that relies on rounding behavior such as $(x + 2^{52}) - 2^{52}$ or ordered floating-point comparisons.

The ULP is defined as follows:

If x is a real number that lies between two finite consecutive floating-point numbers a and b, without being equal to one of them, then $\text{ulp}(x) = |b - a|$, otherwise $\text{ulp}(x)$ is the distance between the two non-equal finite floating-point numbers nearest x. Moreover, $\text{ulp}(\text{NaN})$ is NaN.

7.5 Edge Case Behavior in Flush To Zero Mode

If denormals are flushed to zero, then a function may return one of four results:

1. Any conforming result for non-flush-to-zero mode.
2. If the result given by (1) is a subnormal before rounding, it may be flushed to zero.
3. Any non-flushed conforming result for the function if one or more of its subnormal operands are flushed to zero.
4. If the result of (3) is a subnormal before rounding, the result may be flushed to zero.

In each of the above cases, if an operand or result is flushed to zero, the sign of the zero is undefined.

7.6 Conversion Rules for Floating-point and Integer Types.

The round to zero rounding mode will be used for conversions from a floating-point type to an integer type. The round to nearest even or round to zero rounding mode will be used for conversions from a floating-point or integer type to a floating-point type.

Denormalized half precision floating-point numbers generated when converting a `float` to a `half` or when converting from a `half` to a `float` may be flushed to zero.

When converting a floating-point type to an integer type, the integer value shall be 0 if the floating-point value is NaN.

7.7 Texture Addressing and Conversion Rules

The texture coordinates specified to the `sample`, `sample_compare`, `gather`, `gather_compare`, `read` and `write` functions cannot be INF or NaN. In addition, the texture coordinate must refer to a region inside the texture for the texture `read` and `write` functions.

In the sections that follow, we discuss conversion rules²⁰ that are applied when reading and writing textures in a graphics or kernel function.

7.7.1 Conversion rules for normalized integer pixel data types

In this section we discuss converting normalized integer pixel data types to floating-point values and vice-versa.

7.7.1.1 Converting normalized integer pixel data types to floating-point values

For textures that have 8-bit, 10-bit or 16-bit normalized unsigned integer pixel values, the texture sample and read functions convert the pixel values from an 8-bit or 16-bit unsigned integer to a normalized single or half-precision floating-point value in the range $[0.0 \dots 1.0]$.

For textures that have 8-bit or 16-bit normalized signed integer pixel values, the texture sample and read functions convert the pixel values from an 8-bit or 16-bit signed integer to a normalized single or half-precision floating-point value in the range $[-1.0 \dots 1.0]$.

These conversions are performed as listed in the second column of Table 29. The precision of the conversion rules are guaranteed to be ≤ 1.5 ulp except for the cases described in the third column.

Table 29 Rules for Conversion to a Normalized Float Value

Convert from	Conversion Rule to Normalized Float	Corner Cases
1-bit normalized unsigned integer	<code>float(c)</code>	0 must convert to 0.0 1 must convert to 1.0
2-bit normalized unsigned integer	<code>float(c) / 3.0</code>	0 must convert to 0.0 3 must convert to 1.0

²⁰ The conversion rules described in this section do not apply when a multi-sample resolve operation is performed.

Convert from	Conversion Rule to Normalized Float	Corner Cases
4-bit normalized unsigned integer	$\text{float}(c) / 15.0$	0 must convert to 0.0 15 must convert to 1.0
5-bit normalized unsigned integer	$\text{float}(c) / 31.0$	0 must convert to 0.0 31 must convert to 1.0
6-bit normalized unsigned integer	$\text{float}(c) / 63.0$	0 must convert to 0.0 63 must convert to 1.0
8-bit normalized unsigned integer	$\text{float}(c) / 255.0$	0 must convert to 0.0 255 must convert to 1.0
10-bit normalized unsigned integer	$\text{float}(c) / 1023.0$	0 must convert to 0.0 1023 must convert to 1.0
16-bit normalized unsigned integer	$\text{float}(c) / 65535.0$	0 must convert to 0.0 65535 must convert to 1.0
8-bit normalized signed integer	$\max(-1.0, \text{float}(c)/127.0)$	-128 and -127 must convert to -1.0 0 must convert to 0.0 127 must convert to 1.0
16-bit normalized signed integer	$\max(-1.0, \text{float}(c)/32767.0)$	-32768 and -32767 must convert to -1.0 0 must convert to 0.0 32767 must convert to 1.0

7.7.1.2 Converting floating-point values to normalized integer pixel data types

For textures that have 8-bit, 10-bit or 16-bit normalized unsigned integer pixel values, the texture write functions convert the single or half-precision floating-point pixel value to an 8-bit or 16-bit unsigned integer.

For textures that have 8-bit or 16-bit normalized signed integer pixel values, the texture write functions convert the single or half-precision floating-point pixel value to an 8-bit or 16-bit signed integer.

NaN values are converted to zero.

Conversions from floating-point values to normalized integer values are performed as listed in Table 30.

Table 30 Rules for Conversion from Floating-Point to a Normalized Integer Value

Convert to	Conversion Rule to Normalized Integer
1-bit normalized unsigned integer	$x = \min(\max(f, 0.0), 1.0)$ $i_{0:0} = \text{int}_{\text{RTNE}}(x)$
2-bit normalized unsigned integer	$x = \min(\max(f * 3.0, 0.0), 3.0)$ $i_{1:0} = \text{int}_{\text{RTNE}}(x)$
4-bit normalized unsigned integer	$x = \min(\max(f * 15.0, 0.0), 15.0)$ $i_{3:0} = \text{int}_{\text{RTNE}}(x)$

Convert to	Conversion Rule to Normalized Integer
5-bit normalized unsigned integer	$x = \min(\max(f * 31.0, 0.0), 31.0)$ $i_{4:0} = \text{int}_{\text{RTNE}}(x)$
6-bit normalized unsigned integer	$x = \min(\max(f * 63.0, 0.0), 63.0)$ $i_{5:0} = \text{int}_{\text{RTNE}}(x)$
8-bit normalized unsigned integer	$x = \min(\max(f * 255.0, 0.0), 255.0)$ $i_{7:0} = \text{int}_{\text{RTNE}}(x)$
10-bit normalized unsigned integer	$x = \min(\max(f * 1023.0, 0.0), 1023.0)$ $i_{9:0} = \text{int}_{\text{RTNE}}(x)$
16-bit normalized unsigned integer	$\text{result} = \min(\max(f * 65535.0, 0.0), 65535.0)$ $i_{15:0} = \text{int}_{\text{RTNE}}(x)$
8-bit normalized signed integer	$\text{result} = \min(\max(f * 127.0, -127.0), 127.0)$ $i_{7:0} = \text{int}_{\text{RTNE}}(x)$
16-bit normalized signed integer	$\text{result} = \min(\max(f * 32767.0, -32767.0), 32767.0)$ $i_{15:0} = \text{int}_{\text{RTNE}}(x)$

The GPU may choose to approximate the rounding mode used in the conversions from floating-point to integer value described in the table above. If a rounding mode other than round to nearest even is used, the absolute error of the implementation dependent rounding mode vs. the result produced by the round to nearest even rounding mode must be ≤ 0.6 .

7.7.2 Conversion rules for half precision floating-point pixel data type

For textures that have half-precision floating-point pixel color values, the conversions from `half` to `float` are lossless. Conversions from `float` to `half` round the mantissa using the round to nearest even rounding mode. Denormalized numbers for the `half` data type which may be generated when converting a `float` to a `half` may be flushed to zero. A `float` NaN may be converted to an appropriate NaN or be flushed to zero in the `half` type. A `float` INF must be converted to an appropriate INF in the `half` type.

7.7.3 Conversion rules for single precision floating-point pixel data type

The following rules apply for reading and writing textures that have single-precision floating-point pixel color values.

- NaNs may be converted to a NaN value(s) or be flushed to zero.
- INFs must be preserved.
- Denorms may be flushed to zero.
- All other values must be preserved.

7.7.4 Conversion rules for 11-bit and 10-bit floating-point pixel data type

The floating-point formats use 5 bits for the exponent, 5 bits of mantissa for the 10-bit floating-point types and 6-bits of mantissa for the 11-bit floating-point types with an additional hidden bit for both types. There is no sign bit. The 10-bit and 11-bit floating-point types preserve denorms.

These floating-point formats use the following rules:

- If exponent = 0 and mantissa = 0, the floating-point value is 0.0.
- If exponent = 31 and mantissa != 0, the resulting floating-point value is a NaN.
- If exponent = 31 and mantissa = 0, the resulting floating-point value is positive infinity.
- If $0 \leq \text{exponent} \leq 31$, the floating-point value is $2^{(\text{exponent} - 15)} * (1 + \text{mantissa}/N)$.
- If exponent = 0 and mantissa != 0, the floating-point value is a denormal value given as $2^{(\text{exponent} - 14)} * (\text{mantissa} / N)$

N is 32 if mantissa is 5-bits and is 64 if mantissa is 6-bits.

Conversion of a 11-bit or 10-bit floating-point pixel data type to a half or single precision floating-point value will be lossless. Conversion of a half or single precision floating-point value to a 11-bit or 10-bit floating-point value must be ≤ 0.5 ULP. Any operation that would result in a value less than zero for these floating-point types is clamped to zero

7.7.5 Conversion rules for 9-bit floating-point pixel data type with a 5-bit shared exponent

The RGB9E5_SharedExponent shared exponent floating-point format use 5 bits for the exponent and 9 bits for the mantissa. There is no sign bit.

Conversion from this format to a half or single precision floating-point value will be lossless and is computed as $2^{(\text{shared exponent} - 15)} * (\text{mantissa}/512)$ for each color channel.

Conversion from a half or single precision floating-point RGB color value to this format is performed as follows:

N is the number of mantissa bits per component (9), B is the exponent bias (15) and E_{max} is the maximum allowed biased exponent value (31).

- Components r, g and b are first clamped (in the process, mapping NaN to zero) as follows:

$$r_c = \max(0, \min(\text{sharedexp}_{\text{max}}, r))$$

$$g_c = \max(0, \min(\text{sharedexp}_{\text{max}}, g))$$

$$b_c = \max(0, \min(\text{sharedexp}_{\text{max}}, b))$$

$$\text{where sharedexp}_{\text{max}} = ((2^N - 1)/2^N) * 2^{(E_{\text{max}} - B)}$$

- The largest clamped component max_c , is determined:

$$\text{max}_c = \max(r_c, g_c, b_c)$$

- A preliminary shared exponent exp_p is computed:

$$\text{exp}_p = \max(-B - 1, \text{floor}(\log_2(\text{max}_c))) + 1 + B$$

- A refined shared exponent exp_s is computed:

$$\text{max}_s = \text{floor}((\text{max}_c / 2^{\text{exp}_p - B - N}) + 0.5f)$$

$$\text{exp}_s = \text{exp}_p, \text{ if } 0 \leq \text{max}_s < 2^N, \text{ and}$$

$$= \text{exp}_p + 1, \text{ if } \text{max}_s = 2^N.$$

- Finally, three integer values in the range 0 to $2^N - 1$ are computed:

$$r_s = \text{floor}(r_c / 2^{\text{exp}_p - B - N}) + 0.5f$$

$$g_s = \text{floor}(g_c / 2^{\text{exp}_p - B - N} + 0.5f)$$

$$b_s = \text{floor}(b_c / 2^{\text{exp}_p - B - N} + 0.5f)$$

Conversion of a half or single precision floating-point color values to the RGB9E5 shared exponent floating-point value will be ≤ 0.5 ULP.

7.7.6 Conversion rules for signed and unsigned integer pixel data types

For textures that have 8-bit or 16-bit signed or unsigned integer pixel values, the texture sample and read functions return a signed or unsigned 32-bit integer pixel value. The conversions described in this section must be correctly saturated.

Writes to these integer textures perform one of the conversions listed in Table 31.

Table 31 Rules for Conversion between Integer Pixel Data Types

Convert from	Convert to	Conversion Rule
32-bit signed integer	8-bit signed integer	<code>result = convert_char_saturate(val)</code>
32-bit signed integer	16-bit signed integer	<code>result = convert_short_saturate(val)</code>
32-bit unsigned integer	8-bit unsigned integer	<code>result = convert_uchar_saturate(val)</code>
32-bit unsigned integer	16-bit unsigned integer	<code>result = convert_ushort_saturate(val)</code>

7.7.7 Conversion rules for sRGBA and sBGRA Textures

Conversion from sRGB space to linear space is automatically done when sampling from an sRGB texture. The conversion from sRGB to linear RGB is performed before the filter specified in the sampler specified when sampling the texture is applied. If the texture has an alpha channel, the alpha data is stored in linear color space.

Conversion from linear to sRGB space is automatically done when writing to an sRGB texture. If the texture has an alpha channel, the alpha data is stored in linear color space.

The following is the conversion rule for converting a normalized 8-bit unsigned integer sRGB color value to a floating-point linear RGB color value (call it *c*).

```
if (c <= 0.04045),
    result = c / 12.92;
else
    result = powr((c + 0.055) / 1.055, 2.4);
```

The precision of the above conversion should be such that the delta between the resulting infinitely precise floating point value when `result` is converted back to an un-normalized sRGB value but without rounding to a 8-bit unsigned integer value (call it *r*) and the original sRGB 8-bit unsigned integer color value (call it *r_{orig}*) is ≤ 0.5 i.e.

```
fabs(r - rorig) <= 0.5
```

The following are the conversion rules for converting a linear RGB floating-point color value (call it *c*) to a normalized 8-bit unsigned integer sRGB value.

```
if (isnan(c)) c = 0.0;
if (c > 1.0)
    c = 1.0;
else if (c < 0.0)
    c = 0.0;
else if (c < 0.0031308)
    c = 12.92 * c;
else
    c = 1.055 * powr(c, 1.0/2.4) - 0.055;

convert to integer scale i.e. c = c * 255.0
convert to integer:
    c = c + 0.5
    drop the decimal fraction, and the remaining
    floating-point(integer) value is converted
    directly to an integer.
```

The precision of the above conversion should be such that

```
fabs(reference result - integer result) < 1.0.
```


8 Revision History

Revision History

Version	Date	Notes
1.2	2016-10-21	Updated for iOS 10, tvOS 10, and macOS 10.12



Apple Inc.
Copyright © 2016 Apple Inc.
All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, mechanical, electronic, photocopying, recording, or otherwise, without prior written permission of Apple Inc., with the following exceptions: Any person is hereby authorized to store documentation on a single computer or device for personal use only and to print copies of documentation for personal use provided that the documentation contains Apple's copyright notice.

No licenses, express or implied, are granted with respect to any of the technology described in this document. Apple retains all intellectual property rights associated with the technology described in this document. This document is intended to assist application developers to develop applications only for Apple-branded products.

Apple Inc.
1 Infinite Loop
Cupertino, CA 95014
408-996-1010

Apple is a trademark of Apple Inc., registered in the U.S. and other countries.

APPLE MAKES NO WARRANTY OR REPRESENTATION, EITHER EXPRESS OR IMPLIED, WITH RESPECT TO THIS DOCUMENT, ITS QUALITY, ACCURACY, MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE. AS A RESULT, THIS DOCUMENT IS PROVIDED "AS IS," AND YOU, THE READER, ARE ASSUMING THE ENTIRE RISK AS TO ITS QUALITY AND ACCURACY.

IN NO EVENT WILL APPLE BE LIABLE FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES RESULTING FROM ANY DEFECT, ERROR OR INACCURACY IN THIS DOCUMENT, even if advised of the possibility of such damages.

Some jurisdictions do not allow the exclusion of implied warranties or liability, so the above exclusion may not apply to you.